CENTRO DE BACHILLERATO TECNOLÓGICO INDUSTRIAL Y DE SERVICIOS NO.83

BOOKING RESERVACIONES

ALUMNA: PAREDES CRUZ MARIA FERNANDA

DOCENTE: MARIA VICTORIA CORTÉS GÓMEZ

MATERIA: CONSTRUYE APLICACIONES MÓVILES PARA ANDROID

6 AM2

Índice

Introducción ---------- *3*
Objetivo ---------- *4*
Motivación ---------- *4*
Organización del proyecto ---------- *5*
Requerimientos ---------- *6*
Tecnologías elegidas ---------- *8*
Diseño ---------- *9*
Diseño de pantallas ---------- 10
Estructura de la app ---------- *13*
Recursos utilizados ---------- *14*
Configuración del entorno ---------- 14
Librerías utilizadas ---------- 15
Desarrollo ---------- *17*
Header.js ---------- 17
HomeScreen.js ---------- 18
DateRangePickerComponent.js ---------- 21
CitySearch.js ---------- 24
BookingScreen.js ---------- 27
Login.js ---------- 29
Registration.js ---------- 30
Dashboard.js ---------- 32
Image.js ---------- 33
ProfileUser.js ---------- 35
TakePhoto.js ---------- 39
ResultScreen.js ---------- 40
Calificar.js ---------- 42
Vuelos.js ---------- 46
Ejecución ---------- *49*
Conclusión ---------- *52*
Bibliografía ---------- *53*

Índice de ilustraciones

Ilustración 1 Figma 9
Ilustración 2 Paleta de colores 10
Ilustración 3 Pantalla Principal 11
Ilustración 4 Pantallas Vuelos, Autos, Taxi 11
Ilustración 5 Pantallas favoritos, perfil y reservados 12
Ilustración 6 Estructura 13
Ilustración 7 Importación de recursos 17
Ilustración 8 Botones del stack y estilos 18
Ilustración 9 Componentes HomeScreen 18
Ilustración 10 Estados de HomeScreen 19
Ilustración 11 Actualización de habitaciones 19
Ilustración 12 Confirmación de fechas de reserva 22
Ilustración 13 Pantalla Dashboard 38

Introducción

Actualmente la tecnología ha tomado un papel fundamental para el desarrollo de tareas, ayudando a la simplificación de las mismas empleando procesos automatizados que nos facilitan todo. Las aplicaciones móviles son parte de esta tecnología que se ha convertido en parte fundamental de la vida cotidiana, esto debido a la gran utilidad que presentan al realizar actividades diarias sin necesidad de tanto esfuerzo. Estos programas diseñados para dispositivos como teléfonos o tablets, ofrecen gran variedad de funciones que se puede decir van desde la comunicación con amigos o familiares sin importar la distancia, hasta el entretenimiento y la realización de actividades laborales. Su alta demanda está basada en su capacidad para aprovechar las características de los dispositivos como su conectividad y los sensores integrados que ayudan a brindar una mejor experiencia de usuario.

Existen diferentes tipos de aplicaciones móviles, cada una con sus propias características. Las aplicaciones nativas, por ejemplo, están diseñadas específicamente para un solo sistema operativo, como iOS o Android. Esto permite un rendimiento óptimo y una experiencia de usuario más satisfactoria, ya que son adaptables a las características y funciones del dispositivo.

Por otro lado, las aplicaciones híbridas son aquellas que combinan tecnologías de desarrollo, utilizando principalmente tecnologías web en su núcleo. Estas aplicaciones ofrecen la ventaja de un desarrollo más rápido y una mayor compatibilidad entre plataformas.

Al desarrollar una aplicación, es importante pensar en el tipo de sistema operativo que se utilizará, ya que la elección influye en el rendimiento de la aplicación y en la experiencia que el usuario recibirá. Por decir, las aplicaciones nativas suelen ofrecer un rendimiento óptimo y una experiencia más satisfactoria, mientras que las aplicaciones híbridas pueden desarrollarse más rápidamente y ser compatibles con diferentes dispositivos. Al entender estas características los desarrolladores pueden crear aplicaciones que sean más eficaces, fáciles de usar y que se adapten mejor a las necesidades de los usuarios.

Objetivo

El objetivo principal de desarrollar una aplicación móvil para Android es utilizar los principios de programación aprendidos durante el desarrollo académico, para lograr crear una aplicación funcional y digna para el uso de los usuarios; se busca consolidar los conocimientos adquiridos durante el bachillerato, especialmente en el desarrollo de software y el manejo de bases de datos para almacenar y gestionar la información necesaria para el funcionamiento de la aplicación.

Finalmente, el objetivo es aprobar el módulo de creación de aplicación móviles para Android con el desarrollo de una aplicación completa y eficiente para su uso.

Motivación

La motivación detrás de la creación de una aplicación móvil centrada en la reserva de hospedajes surge del deseo de finalizar la formación técnica con un proyecto bien elaborado. Durante el tiempo en la institución, se han adquirido diversas habilidades y conocimientos en programación, diseño web y gestión de proyectos. Por lo tanto, se ve este proyecto como una oportunidad para aplicar todo lo aprendido y demostrar la capacidad técnica desarrollada día a día como programador.

Organización del proyecto

Este proyecto de aplicación móvil involucra una estrategia de navegación utilizando dos tipos principales de navegación: el Stack Navigator y el BottomTabNavigator.

El Stack Navigator se emplea para gestionar la navegación entre las distintas pantallas de la aplicación de una manera jerárquica. Al adoptar esta técnica, se establece un flujo que permite a los usuarios moverse hacia adelante y hacia atrás entre las diferentes secciones de la aplicación de manera sencilla.

Por otro lado, el BottomTabNavigator se utiliza para proporcionar acceso rápido y directo a las funciones principales de la aplicación desde la parte inferior de la pantalla. Esta barra de pestañas permite a los usuarios cambiar fácilmente entre las distintas secciones de la aplicación con solo tocar un ícono, lo que agiliza la navegación de la aplicación.

Cada una de estas formas de navegación dirige a los usuarios a cuatro páginas principales para la ejecución de la app:

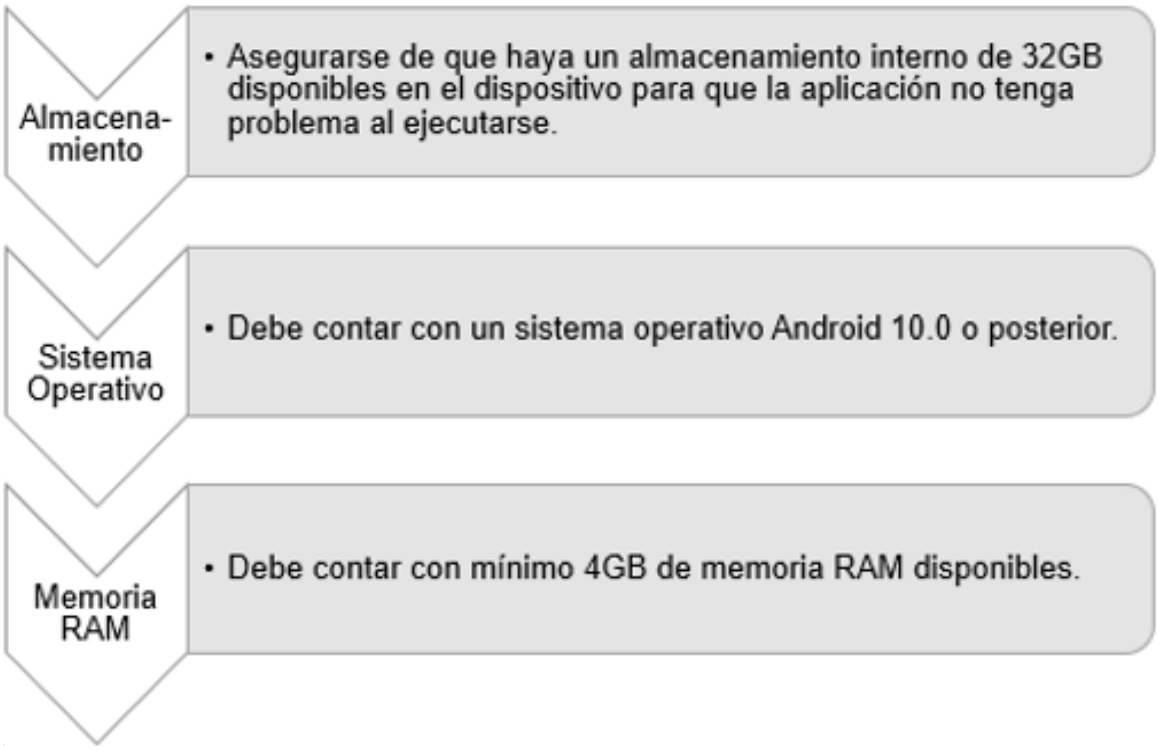
BookingScreen	• Esta pantalla ofrece una visualización de las opciones de hospedaje disponibles. Aquí, los usuarios pueden explorar y seleccionar entre una variedad de alojamientos según sus preferencias y necesidades.
HomeScreen	• La página principal de la aplicación, es la primera vista que tienen los usuarios al ingresar. Desde aquí, pueden acceder a todas las funciones y características principales de la aplicación.
ProfileScreen	• Esta pantalla está dedicada a la administración del perfil del usuario que está usando la aplicación. Aquí, los usuarios pueden ver y editar su información personal, ajustar sus preferencias o bien, crear un nuevo perfil.
SavedScreen:	• Esta pantalla alberga los alojamientos favoritos que los usuarios han guardado para acceder rápidamente a ellos cuando así lo necesiten. Esta función permite a los usuarios mantener una lista de sus opciones preferidas y realizar reservaciones rápidamente.

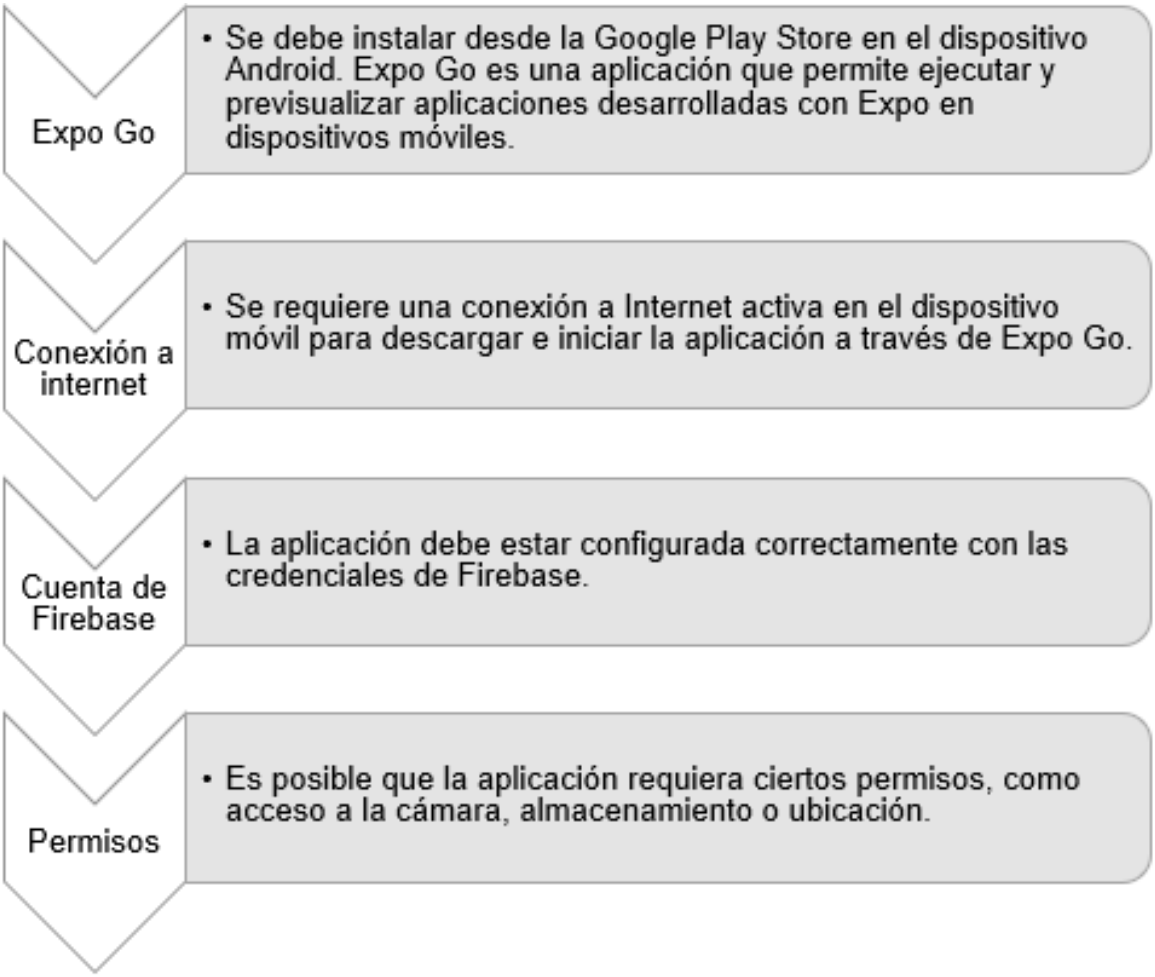
Esta estructura de navegación y diseño de pantallas organiza las funciones de manera lógica y accesible mejorando la facilidad de uso de la aplicación. Con la incorporación de distintas pantallas se tiene un mejor control sobre el contenido que se mostrará en cada una de ellas, para que así los usuarios puedan utilizar la aplicación fluidamente y puedan encontrar lo que buscan sin ningún problema.

Requerimientos

Es importante tomar en cuenta los requerimientos del dispositivo al desarrollar una aplicación móvil, ya que garantiza que la app sea compatible con el dispositivo, optimizando su rendimiento y adaptando sus funcionalidades a las capacidades específicas del teléfono.

Así pues, podemos minimizar el uso de recursos, aprovechar las características únicas del dispositivo y reducir el impacto en el almacenamiento. Para garantizar que la aplicación se ejecute correctamente en un dispositivo de sistema operativo Android debemos tener en cuenta algunos de los siguientes puntos:





Es fundamental saber cuáles son los requerimientos necesarios para que una aplicación funcione correctamente en cualquier dispositivo, ya que así el desarrollador se puede asegurar de que los usuarios no tengan ningún problema al utilizarla. Además, al conocer los factores como la versión del sistema operativo, los recursos de hardware y las especificaciones técnicas, mejora el rendimiento de la aplicación para que no se detenga inesperadamente y se ejecute sin interrumpir la experiencia que el usuario está teniendo.

Tecnologías elegidas

En la implementación de la aplicación móvil, se han seleccionado las tecnologías que mejor se ajustan a las necesidades y objetivos. React Native, por ejemplo, ofrece una manera eficiente de crear interfaces de usuario dinámicas. Expo proporciona herramientas y servicios que se encargan de la gestión en dispositivos móviles permitiendo ver la aplicación desde un dispositivo físico. Finalmente, Firebase ofrece una solución completa de backend para la autenticación de usuarios, el almacenamiento de datos en tiempo real y la gestión de notificaciones push.

React Native

- Es un marco de desarrollo de código abierto creado por Facebook que se utiliza para construir aplicaciones móviles multiplataforma utilizando JavaScript y React. Permite a los desarrolladores utilizar un solo código base, ahorrando tiempo y recursos ya que utiliza componentes reutilizables que pueden ser compartidos entre diferentes partes de la aplicación.
- La interfaz de usuario se renderiza utilizando componentes nativos especificos de cada plataforma, además incluye una función Hot reloading que permite a los desarrolladores ver los cambios en tiempo real.

Firebase

- Es una plataforma de desarrollo de aplicaciones móviles y web desarrolladas por Google, ofrece servicios y herramientas para ayudar a los desarrolladores a construir sus aplicaciones, especialmente en el uso de base de datos.
- Firebase RealTime Database es una base de datos en tiempo real alojada en la nube que permite almacenar y sincronizar datos, imágenes, videos y documentos entre los usuarios en tiempo real, es una base de datos NoSQL que hace que los cambios realizados se reflejan instantáneamente en todos los dispositivos conectados.

Expo

- Es una plataforma gratuita y de código abierto que simplifica el proceso de desarrollo de aplicaciones móviles utilizando React Native, proporciona una serie de herramientas y servicios que permiten a los desarrolladores construir, probar y distribuir aplicaciones móviles de forma rápida y eficiente.

- Ofrece herramientas CLI (Command Line Interface) que simplifican el proceso de inicio de un nuevo proyecto y la getsion de dependencias. Incluye una biblioteca de componentes UI predefinidos que pueden ser integrados fácilmente en aplicaciones móviles.

La combinación de React Native, Expo y Firebase proporciona un conjunto de herramientas que permiten crear una aplicación escalable y fácil de usar, mientras se mantenga un enfoque en el que la prioridad es la experiencia de los usuarios. Al aprovechar al máximo estas tecnologías, se puede crear una aplicación que cumpla con un estándar propios del desarrollador en el que lo primordial es la funcionalidad, el diseño y el rendimiento.

Diseño

El diseño de una aplicación requiere una minuciosa atención, ya que constituye la primera impresión que los usuarios tienen de la misma. A menudo, las personas se ven influenciadas por la apariencia visual de las cosas, incluso antes de preocuparse por su funcionalidad.

Un diseño cuidadosamente elaborado facilita la navegación de los usuarios por la aplicación. Si el diseño es atractivo y llamativo puede transmitir una sensación de confianza y seguridad hacia el usuario. Para lograr que el diseño sea interesante para los consumidores de la aplicación se hará uso de un recurso conocido como Figma.

Ilustración 1
Figma

Figma es una herramienta que ayuda en el diseño de interfaces UX y UI. Cuenta con distintos tipos de proyectos o plantillas para ayudar con el diseño

de las pantallas o bien también tiene un espacio para crear un proyecto desde cero. Permite agregar texto, imágenes e iconos para un mejor resultado. Para el desarrollo de la aplicación se utilizará de tal manera que muestre prototipos de las pantallas antes de diseñarlas mediante código.

Para dar una mejor vida a la aplicación de hará uso de una paleta de colores que combina el verde oliva, verde oscuro, marrón claro y naranja brillante debido a las sensaciones que se buscan transmitir a los usuarios.

Verde: Los tonos verdes se utilizan debido a que se busca darle una apariencia de elegancia y sofisticación a la aplicación para que el usuario confié en los servicios que ofrece, igualmente al ser este un color que se asocia mucho a lo que es la naturaleza se busca transmitir tranquilidad.

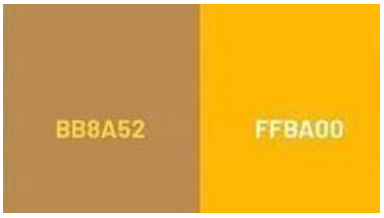


Ilustración 2 Paleta de colores

Marrón: Transmite una sensación de calidez haciendo que los usuarios se sientan cómodos al utilizar la aplicación.

Naranja: Este al ser un color enérgico le agrega vida a la aplicación, a la vez que hace que se vea más dinámica. Este color será utilizado para resaltar aspectos importantes en la aplicación.

Diseño de pantallas

El diseño de las interfaces de usuario no solo se trata de la estética visual, sino también de la usabilidad, la accesibilidad y la capacidad de una navegación intuitiva de la aplicación. A continuación, se presenta un boceto de las pantallas y se describe un poco el uso que se les dará.

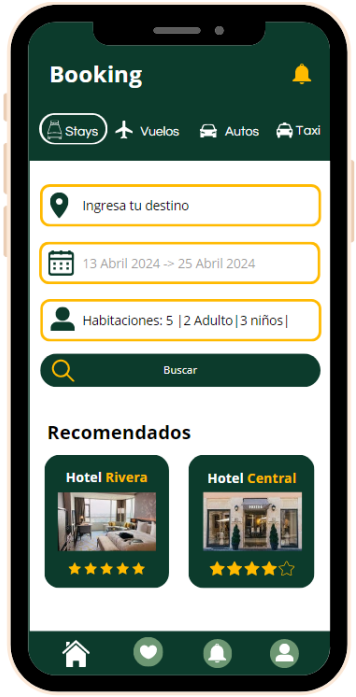


Ilustración 3 Pantalla Principal

Contiene tres contenedores de texto para que el usuario pueda colocar su preferencia de destino, las fechas que quiere reservar y el tipo de habitaciones.

Además, se añade una recomendación de los hoteles mejor puntuados por el usuario. Para esto se utiliza una secuencia de imágenes en slider y la puntuación que ofrece la mayoría de los usuarios

Esta pantalla utiliza la ubicación y hace uso de un calendario.

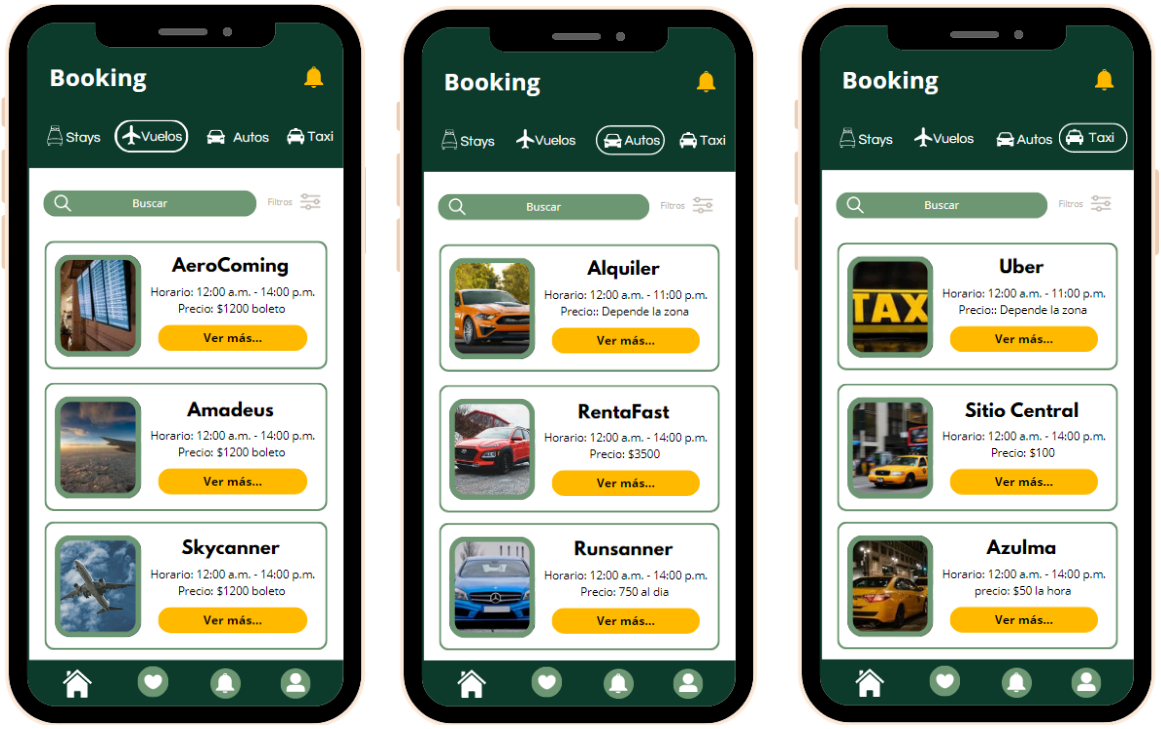


Ilustración 4 Pantallas Vuelos, Autos, Taxi

Las pantallas de vuelos, autos y taxi son para que el usuario elija sus preferencias de cada cosa, para esto contienen una barra de búsqueda y filtros que permiten seleccionar especificaciones, por ejemplo, si quiere un vuelo comercial o privado, un rango de precios o fechas específicas.

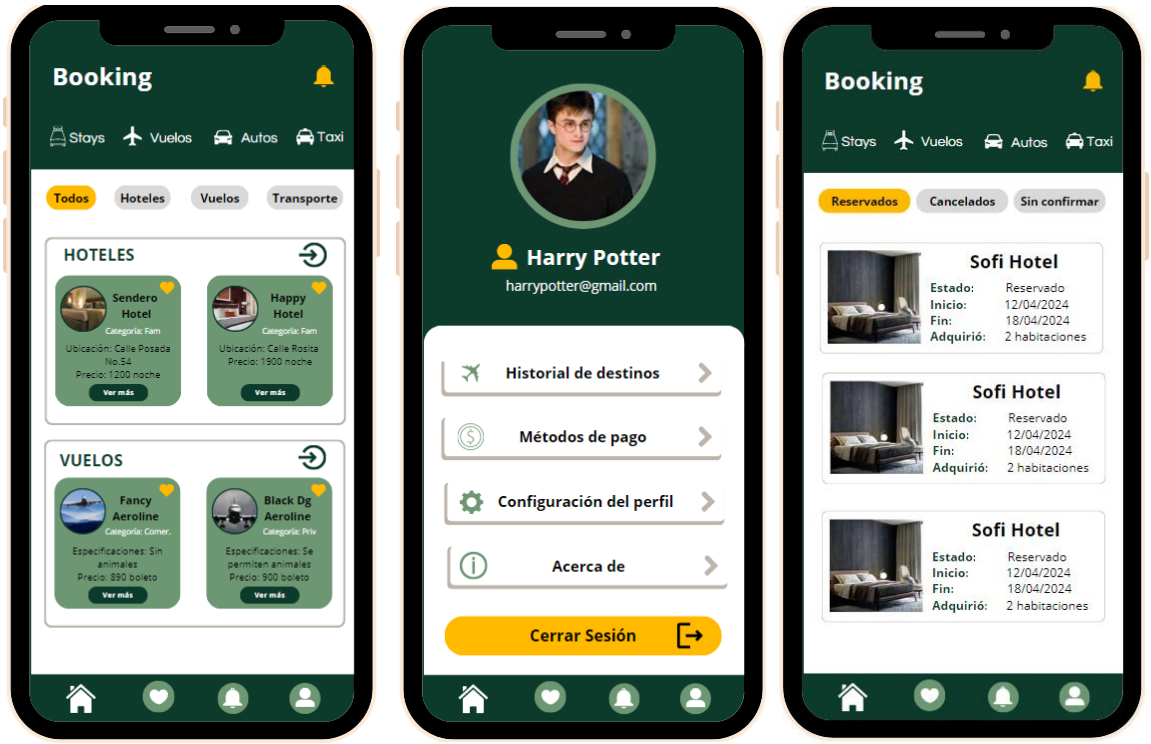


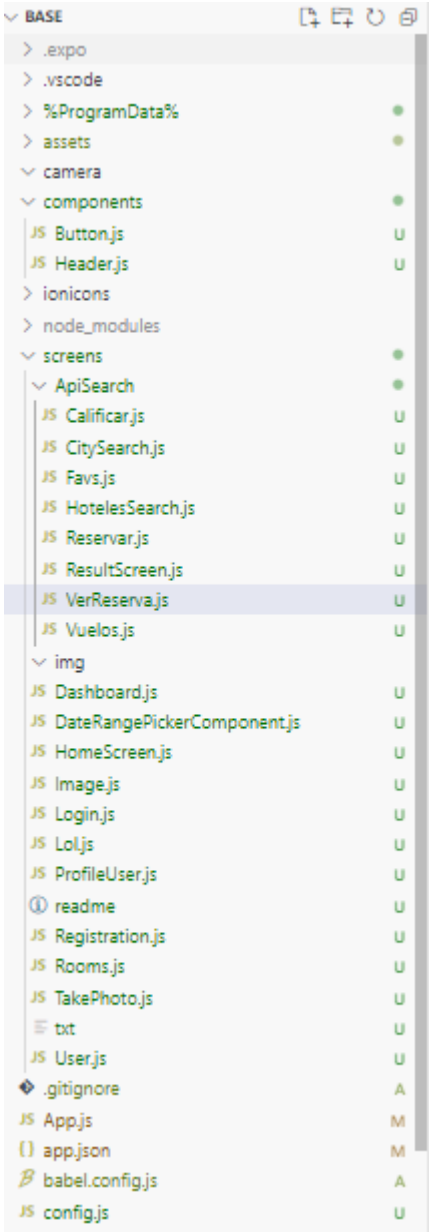
Ilustración 5 Pantallas favoritos, perfil y reservados

En la primera pantalla se muestran los favoritos de cada usuario de acuerdo a secciones, esto con el objetivo de que pueda recurrir a sus preferencias cada que desee hacer un viaje. Dentro de la segunda pantalla el usuario puede editar su perfil de usuario, ver sus métodos de pago guardados previamente y también puede consultar información acerca de la app. Y finalmente en la última pantalla puede ver los hoteles con las que tuvo alguna interacción.

El diseño visual influye en la primera impresión que los usuarios tienen de la aplicación, su facilidad de uso, legibilidad y estética general. Una paleta de colores bien elegida puede transmitir emociones, mejorar la comprensión de la información y crear una mejor experiencia para los usuarios.

Estructura de la app

Para comenzar a desarrollar la aplicación es esencial contar con una estructura bien organizada dentro del entorno de desarrollo (Visual Studio Code) ya que esto facilita el mantenimiento de la aplicación a medida que crece y permite ubicar los archivos rápidamente y actualizar partes específicas del código.



También ayuda a la escalabilidad del proyecto, ya que una estructura bien diseñada facilita la integración de nuevas características y la expansión de las funciones sin generar problemas en otros archivos. A continuación, se tiene una imagen que muestra cómo es que se tiene organizada la aplicación hasta el momento.

Assets: Dentro de esta carpeta se encuentran las imágenes que se utilizarán para una mejor vista de la aplicación.

Componentes: Dentro de esta carpeta se tienen los componentes reutilizables, es decir aquellos fragmentos de código que se utilizan en todas las pantallas, esto con el objetivo que no haya repetición de código.

ApiSearch: Contiene las pantallas que buscan los datos de los hoteles y recuperan los datos de Firebase.

Screens: Guarda todos los archivos JavaScript que contienen el código de las pantallas que conforman la aplicación. Dentro de esta carpeta se encuentra el archivo BookingScreen, HomeScreen, ProfileScreen, Login y todas las que funcionan como un primer vistazo de la aplicación.

También está la carpeta *node_modules* que incluye todas las librerías de node que se instalan para que la aplicación funcione correctamente.

Además de estas carpetas también se cuenta con dos archivos Js adicionales como lo son App.js que sirve para incluir las rutas de navegación y demás, es la que hace que la aplicación se ejecute y el archivo StackNavigator.js que incluye el código del Stack y de las BottomTabs.

Se debe tener una buena organización para que la legibilidad del código de la aplicación no se vea afectada, permitiendo comprender más fácilmente la lógica y la estructura del proyecto. Asimismo, la organización de carpetas distintas aumenta la eficiencia ya que se reduce el tiempo al buscar archivos específicos a los que se les requiere hacer alguna modificación.

Recursos utilizados

Es importante mencionar los recursos que se utilizan en la aplicación, ya que de esta forma podemos demostrar que nadie más ha creado la aplicación, lo que puede hacer que los usuarios confíen más en utilizarla. Además, es importante saber que recursos utilizamos e irlos documentando ya que esto puede ser útil por si en el futuro la aplicación presenta problemas de implementación saber cómo tratarlos.

Configuración del entorno

La configuración del entorno es el primer paso al crear una aplicación web, es incluso el más importante ya que en caso de no llevarlo a cabo bien la aplicación puede presentar problemas al ejecutarse, o bien, ni siquiera llegar a hacerlo. Esta es la base sólida para empezar a desarrollar ya que permite crear una primera plantilla en blanco para continuar construyendo la aplicación.

1: Primero que todo se debe de instalar Node.js, que es el entorno que nos permite ejecutar código JavaScript fuera de un navegador.

2: Se abre en CMD y se escribe el comando node -v para saber si ya tenemos instalado Node.

```
PS C:\Users\usuario> node -v
v20.11.0
```

3: Ejecutar el comando para saber que versión de npm se tiene instalada. En caso de tener una versión menor a la 10.4.0 ejecutar el comando *npm install -g @10.4.0*

```
PS C:\Users\usuario> npm -v
10.4.0
```

4: Instalar expo-cli, el cual utilizaremos para facilitar el desarrollo de la aplicación

```
npm install --global expo-cli
```

5: Para crear la plantilla del proyecto, se tiene tres formas distintas de escribir el comando, sin embargo, todas cumplen la misma función.

```
npm create react-native-app booking
```

```
npx create-expo-app booking
```

```
expo init booking
```

6: Una vez que se tiene creado el proyecto se procede a abrirlo mediante comando.

```
C:\Users\usuario> cd booking
C:\Users\usuario\booking>
```

Cuando se terminó de configurar la plantilla para desarrollar la página web, solo queda ir instalando las dependencias que se utilizarán para mejorar el proyecto como la dependencia para la navegación o la de iconos y que se mencionan en el siguiente apartado. Para su instalación es importante saber que se puede utilizar el comando *npm i* o bien *npm install*.

Librerías utilizadas

Se puede decir que las librerías son paquetes de código que son reutilizables, es decir, contiene funciones, componentes y recursos que son definidos para facilitar su uso al momento de desarrollar.

Librerías	Utilidad
expo/vector-icons	Esta librería ayuda a importar los iconos vectoriales proporcionados por Expo.

react-native-date-ranges	Proporciona un calendario interactivo que permite al usuario elegir entre un rango de fechas.
react-native-modals	Esta librería proporciona cierta parte de contenedores que sobresalen de la interfaz, es decir, se utiliza para agregar ventanas emergentes al seleccionar algo.
react-navigation/native	Permite la navegación entre diferentes pantallas, ofreciendo una variedad de navegadores y configuraciones para utilizar cualquier forma de navegar.
react-native-screens	Ayuda a mejorar la fluidez de la transición entre pantallas, actúa en conjunto con la librería anterior.
react-native-safe-area-context	Permite ajustar los componentes al tamaño de la pantalla, haciendo que se adapten y presenten una armonía visual.
react-navigation/native-stack	Proporciona un navegador entre pantallas, es decir, organiza las pantallas para que la navegación se pueda llevar a cabo.
react-navigation/bottom-tabs	Permite llevar a cabo la navegación mediante componentes que se ubican en la parte inferior de la pantalla.
Axios	Esta librería ayuda a recibir y gestionar URL de tipo HTTPS, que se generan para acceder a información de internet.
ImagePicker	Se utiliza para poder acceder a las imágenes de la galería del usuario.
Camera	Se usa para el funcionamiento de la cámara.

El uso de librerías para React Native es importante porque tienen un impacto significativo en el desarrollo de aplicaciones. Estas herramientas representan una parte que influye en la eficiencia, la calidad y la experiencia del usuario final. El uso de librerías permite a los desarrolladores aprovechar componentes reutilizables en lugar de tener que desarrollarlos desde cero. Las dependencias proporcionan soluciones probadas y confiables para tareas como la gestión del estado, la navegación o la autenticación.

Desarrollo

El siguiente apartado será para la explicación de la programación, es decir se habla de manera clara y concisa como es que se realizó el código y para que funciona cada cosa que se utiliza.

Header.js

Esta pantalla se encarga de generar el stack superior que se utiliza para la navegación. Dependiendo del botón presionado es que dirigirá a una página distinta ya preestablecida.

La primera parte se encarga de importar los elementos necesarios para construir el stack. Además del useNavigation que hará posible la navegación.

```
import React from 'react'
import { Pressable, Text, View } from 'react-native'
import { Ionicons } from '@expo/vector-icons';
import { useNavigation } from '@react-navigation/native';

const Header = () => {
  const navigation = useNavigation();
  const goToSavedScreen = () => {
    navigation.navigate('SavedScreen');
  };
```

Ilustración 7 Importación de recursos

Header es la función que guarda todo el código para importarlo. Después de esto se establece un objeto navigation para ir de pantalla a otra y finalmente la función goToSavedScreen hace uso de este objeto. Todos los botones tienen una navegación a una pantalla distinta.

En otra parte del código está el return, dentro de este se establece parte de la interfaz del stack, aquí se encuentra el código de los botones con sus

características, y justo debajo se muestra el style que define el estilo de los botones. Finalmente, en la última línea se hace la exportación de la función Header.

```
return (
  <View style={styles.Vista1}>
    <Pressable style={styles.Boton1}>
      <Ionicons name="bed" size={26} color="white" />
      <Text style={styles.Texto}>Stays</Text>
    </Pressable>

    <Pressable onPress={goToSavedScreen} style={styles.Boton2}> ···
    </Pressable>

    <Pressable style={styles.Boton2}> ···
    </Pressable>

    <Pressable style={styles.Boton2}> ···
    </Pressable>
  </View>
)
```

```
const styles = StyleSheet.create({
  Vista1: {
    backgroundColor: "#0C3B2E", height: 65, flexDirection: "row",
    alignItems: "center", justifyContent: "space-around",
  },
  Boton1: {
    flexDirection: "row", alignItems: "center", borderColor: "white",
    borderWidth: 1, borderRadius: 25, padding: 8,
  },
  Boton2: {
    flexDirection: "row", alignItems: "center",
  },
  Texto: {
    marginLeft: 8, fontWeight: "bold", color: "white",
    fontSize: 15,
  }
})
export default Header
```

Ilustración 8 Botones del stack y estilos

HomeScreen.js

Esta es la página principal que ve el usuario a la aplicación. Lo primero que vemos del código son las librerías de navegación, de iconos y la importación de componentes para crear la interfaz. También se aprecia la integración de la pantalla header, la cual se utilizará en todas las pantallas principales pues se puede decir cumple la función de un menú.

```
import { useNavigation, useRoute } from '@react-navigation/native';
import React, { useLayoutEffect, useState, useEffect } from 'react';
import { Text, StyleSheet, View, ScrollView, Pressable, Button, TextInput } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import Header from '../componentes/Header';
import DatePicker from 'react-native-date-ranges';
import { BottomModal } from "react-native-modals";
import { ModalFooter } from "react-native-modals";
import { ModalButton } from "react-native-modals";
import { ModalTitle } from "react-native-modals";
import { SlideAnimation } from "react-native-modals";
import { ModalContent } from "react-native-modals";
```

Ilustración 9 Componentes HomeScreen

Comenzando con lo que es el estado de la pantalla, se utilizan hooks de estado dados por useState para mantener el estado de la pantalla, como las fechas seleccionadas, el número de habitaciones y la ubicación.

```
const HomeScreen = () => {
    const navigation = useNavigation();
    const route = useRoute();
    const [selectedDates, setSelectedDates] = useState();
    const [rooms, setRooms] = useState(1);
    const [adults, setAdults] = useState(1);
    const [children, setChildren] = useState(0);
    const [modalVisibile, setModalVisibile] = useState(false);
    console.log(selectedDates);
    const selectedCity = route.params?.selectedCity || "Ciudad no seleccionada";
    const handleSearch = () => {
        navigation.navigate('ResultScreen', { rooms, startDate: selectedDates.startDate, endDate: selectedDates.endDate, cityName: selectedCity.name });
    };
```

Ilustración 10 Estados de HomeScreen

```
useLayoutEffect(() => {
    navigation.setOptions({
        headerShown: true,
        title: "Booking",
        headerTitleStyle: {
            fontSize: 20,
            fontWeight: "bold",
            color: "white",
            textAlign: "center",
        },
        headerStyle: {
            backgroundColor: "#0C3B2E",
            height: 110,
            borderBottomColor: "transparent",
            shadowColor: "transparent",
        },
        headerRight: () => (
            <Ionicons name="notifications" size={24}
                style={{ marginRight: 12 }}
            />
        )
    })
}, []);
```

Para dar una mejor presentación a la pantalla, se utiliza useLayoutEffect para configurar el encabezado de la navegación con un título y un ícono de notificación. Las opciones de navegación incluyen:

headerShown: Un valor booleano que indica si se debe mostrar el encabezado.

title: El título del encabezado.

headerTitleStyle: Un objeto que define el estilo del título del encabezado.

headerRight: Un componente que se mostrará en el lado derecho del encabezado. En este caso, se utiliza un ícono de notifications.

```
const updateRooms = () => {
    const totalPersons = (adults + children) + 1;
    setRooms(totalPersons);
};

const updateRooms2 = () => {
    const totalPersons = rooms - 1;
    if (totalPersons < 0) {
        totalPersons = 0;
    }
    setRooms(totalPersons);
};
```

Ilustración 11 Actualización de habitaciones

Las funciones de update rooms se utilizan para cambiar los valores cuando el usuario este interactuando con la aplicación y decidiendo acerca de cuantas habitaciones solicita reservar.

```
<View >
    <Text >Niños</Text>
    <Pressable >
        <Pressable
            onPress={() => {
                setChildren(Math.max(0, children - 1));
                updateRooms2();

            }}>
            <Text >-</Text>
        </Pressable>
        <Pressable>
            <Text>{children}</Text>
        </Pressable>

        <Pressable
            onPress={() => {
                setChildren((c) => c + 1)
                updateRooms();
            }}
            >
            <Text>+</Text>
        </Pressable>
    </Pressable>
</View>
</ModalContent>
</BottomModal>
```

El modal de selección de habitaciones se implementa un modal que permite al usuario seleccionar el número de habitaciones de adultos y niños. Este modal se muestra al presionar un botón de selección en la pantalla principal y aparecen dos opciones que permiten cambiar el estado de las variables children y adults.

Dentro del botón para seleccionar la ciudad se hace uso de un operador ternario que muestra un texto o un campo de entrada de texto dependiendo de si selectedCity está definido o no. Si selectedCity está definido, muestra el nombre de la ciudad seleccionada. De lo contrario, muestra un campo de entrada de texto para que el usuario ingrese el destino. Para que se pueda realizar la búsqueda con ayuda de la API de Amadeus, al presionar el botón se abre otra pantalla la cual permitirá la introducción de datos.

```
<Pressable
    onPress={() => navigation.navigate("Search")}
    style={{ ···
    }}>
    <Ionicons name="location-sharp" size={24} color="#0C3B2E" />
    {selectedCity ? (
        <Text>{selectedCity.name}</Text>

    ) : (
        <TextInput
            placeholderTextColor="black"
            placeholder="Ingresa tu destino"
        />
    )}
</Pressable>
```

DateRangePickerComponent.js

Dentro de esta pantalla se escribe el código que se integra en la pantalla HomeScreen para que el calendario se pueda ver. Lo primero que se hace es la importación de librerías, en este caso se usa la librería react-native-calendars, que da los componentes Calendar y LocaleConfig para poder configurar el calendario y sus fechas.

```
import React, { useState } from "react";
import { View, TouchableOpacity, StyleSheet, Text, Alert } from "react-native";
import { Calendar, LocaleConfig } from "react-native-calendars";
import { useNavigation } from "@react-navigation/native";

LocaleConfig.locales["es"] = {
  monthNames: [
    "Enero","Febrero","Marzo","Abril",
    "Mayo","Junio","Julio","Agosto",
    "Septiembre","Octubre","Noviembre",
    "Diciembre",
  ],
  monthNamesShort: [
    "Ene.","Feb.", "Mar.",
    "Abr.", "May.","Jun.",
    "Jul.","Ago.","Sep.",
    "Oct.", "Nov.","Dic.",
  ],
  dayNames: [
    "Domingo", "Lunes",
    "Martes", "Miércoles",
    "Jueves", "Viernes",
    "Sábado",
  ],
  dayNamesShort: ["Dom.", "Lun.", "Mar.", "Mié.", "Jue.", "Vie.", "Sáb."],
};

LocaleConfig.defaultLocale = "es";
```

El LocaleConfig configura la localización para que el calendario muestre los nombres de los meses y días en español.

El componente DateRangePickerComponent es el que se exportará para su uso, recibe un onConfirm debido a que con esto se sabe si la confirmación del rango de fechas fue aceptada o no. Las demás constantes se utilizan para guardar los estados de la fecha inicial, la fecha final y el hook de navegación.

```
const DateRangePickerComponent = ({ onConfirm }) => {
  const [startDate, setStartDate] = useState(null);
  const [endDate, setEndDate] = useState(null);
  const navigation = useNavigation();
```

```
const onDayPress = (day) => {
  if (!startDate || endDate) {
    setStartDate(day.dateString);
    setEndDate(null);
  } else if (day.dateString < startDate) {
    setStartDate(day.dateString);
  } else {
    setEndDate(day.dateString);
  }
};
```

OnDayPress es una función que se ejecuta cada vez que el usuario selecciona un día en el calendario. Day es el objeto que representa el día seleccionado y es un string que contiene la fecha en formato YYYY-MM-DD. El primer if indica si hay una fecha de inicio seleccionada y lo convierte en string, el segundo else if verifica si la fecha

seleccionada es anterior a la fecha de inicio y se actualiza la fecha de inicio. Finalmente, el ultimo bloque se ejecuta si ninguna de las condiciones anteriores es verdadera y asigna el valor a la fecha final.

La función HandleSelectRange envía una alerta al usuario cuando ya se tiene seleccionado el rango de fechas, para que el usuario confirme los datos seleccionados, en caso de que no se seleccionen se regresa a la parte del calendario y si se selecciona se envían los datos a la pantalla HomeScreen.

```
const handleSelectRange = () => {
  if (startDate && endDate) {
    Alert.alert(
      "Confirmación",
      `¿Desea seleccionar el rango de fechas del ${startDate} al ${endDate}?`,
      [
        {
          text: "No",
          onPress: () => console.log("Selección cancelada"),
          style: "cancel",
        },
        {
          text: "Sí",
          onPress: () => {
            console.log("Rango seleccionado:", startDate, "a", endDate);
            onConfirm(startDate, endDate);
            navigation.navigate('HomeScreen');
          },
        },
      ]
    );
  } else {
    Alert.alert("Error", "Por favor seleccione un rango válido de fechas.");
  }
};
```

Ilustración 12 Confirmación de fechas de reserva

```
const markedDates = {};

if (startDate) {
  markedDates[startDate] = {
    startingDay: true,
    color: "#BB8A52",
    textColor: "white",
  };
}

if (endDate) {
  markedDates[endDate] = {
    endingDay: true,
    color: "#BB8A52",
    textColor: "white",
  };
}
```

markedDates es un objeto que se utilizará para almacenar las fechas y sus propiedades. Este objeto será pasado al componente del calendario para indicar qué fechas están seleccionadas y darles estilo.

Ambas condiciones revisan si el estado (startDate o endDate) tienen algún valor y de ser así se agrega una entrada en el objeto markedDates para indicar que tipo de fecha es, el color de fondo dentro del calendario y cambiar el color del texto de la fecha.

En la siguiente parte de código, se verifica si el rango de fechas ya está completo, de ser así crea una nueva fecha que se inicializa con la fecha inicial y que se usara para iterar desde esta fecha hasta la final. El bucle while se ejecuta hasta que el currentDate sea menor o igual que la fecha final y permite recorrer todas las fechas desde la inicial hasta la última y al final se convierte a un formato de cadena para poder usar markedDates y cambiar el color de fondo y texto dependiendo en donde empieza y en donde termina.

```
if (startDate && endDate) {
  let currentDate = new Date(startDate);
  while (currentDate <= new Date(endDate)) {
    const dateString = currentDate.toISOString().split("T")[0];
    if (dateString === startDate || dateString === endDate) {
      markedDates[dateString] = {
        ...markedDates[dateString],
        color: "#BB8A52",
        textColor: "white",
      };
    } else {
      markedDates[dateString] = { color: "#FFBA00", textColor: "black" };
    }
    currentDate.setDate(currentDate.getDate() + 1);
  }
}

console.log("markedDates:", markedDates);
```

```
return (
  <View>
    <Text style={styles.title}>Elige un rango de fechas</Text>
    <Calendar
      current={new Date().toISOString().split("T")[0]}
      onDayPress={onDayPress}
      markedDates={markedDates}
      markingType={"period"}
    />
    <TouchableOpacity
      title="Seleccionar fecha"
      onPress={handleSelectRange}
      style={styles.boton}
    >
      <Text style={styles.botonText}>Seleccionar</Text>
    </TouchableOpacity>
  </View>
);
;
```

Este fragmento define la estructura del componente DateRangePickerComponent. Dentro de un contenedor View, se presenta un calendario que muestra la fecha actual y permite seleccionar fechas que se resaltan utilizando el objeto markedDates. Debajo del calendario, hay un botón que al ser presionado ejecuta la función handleSelectRange.

CitySearch.js

En esta pantalla es donde se hace la búsqueda de lugares disponibles. La importación especial que se incluye en esta pantalla y en cualquiera de las que haga búsquedas es axios, usado para realizar solicitudes HTTP, lo que permite la comunicación con un servidor remoto para obtener o enviar datos.

```
import React, { useState, useEffect } from 'react';
import { View, Text, FlatList, TouchableOpacity, ActivityIndicator,
  StyleSheet, Pressable, Image, TextInput, Button } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation } from '@react-navigation/native';
import axios from 'axios';
```

Los hooks de estados a utilizar en la pantalla son los que se muestran en la imagen.

```
const CitySearch = () => {
  const navigation = useNavigation();
  const [searchKeyword, setSearchKeyword] = useState('');
  const [imageURL1, setImageURL1] = useState(null);
  const [imageURL2, setImageURL2] = useState(null);
  const [flightDestinations, setFlightDestinations] = useState([]);
  const [showMasInfo, setShowMasInfo] = useState(false);
  const [selectedFlight, setSelectedFlight] = useState(null);
  const [loading, setLoading] = useState(true);
```

searchKeyword se utiliza para almacenar la palabra que ingrese el usuario en el input.

imageURL1 guarda la url de la imagen a mostrar, la cual igualmente es generada por una API.

flightDestinations almacena la lista de todos los resultados.

showMasInfo abre una página para ver más información de la ciudad y elegirla.

SelectedFlight almacena los datos de cada ciudad y loading sirve para mostrar si la búsqueda se está realizando o hay alguna falla.

```
useEffect(() => {
  fetchFlightDestinations();
  fetchCityImages();
}, []);
```

Para realizar la búsqueda de ciudades se hace uso de un hook de efecto. Este efecto secundario se ejecuta cuando el componente se monta por primera vez debido al arreglo de dependencias vacío ([]). Dentro del efecto, se llaman a las funciones fetchFlightDestinations() y fetchCityImages(), que se encargan de obtener datos de las ciudades y cargar las imágenes de las ciudades.

La siguiente parte del código muestra como se hace búsqueda de imágenes por medio de una API, para no ocupar tanto espacio de almacenamiento con las imágenes guardadas.

```
const fetchCityImages = async () => {
  try {
    const response1 = await axios.get('https://api.unsplash.com/photos/random', {
      params: {
        query: 'ciudad',
        orientation: 'landscape',
        client_id: 'ypGbeZFXaNXsiHP77Wk41gcerLfkx1BnsnDBi76vVws'
      }
    });
    const imageURL1 = response1.data.urls.regular;
    setImageURL1(imageURL1);
    setLoading(false);
  } catch (error) {
    console.error('Error fetching city image:', error);
    setLoading(false);
  }
};
```

Primero se define una función asíncrona llamada fetchCityImages. Se utilizan dos llamadas a la función axios.get para realizar solicitudes HTTP a la API de Unsplash y obtener imágenes aleatorias. Cada solicitud se hace a la misma URL especificando que se desean imágenes relacionadas con la palabra clave "ciudad".

Después de obtener las respuestas de las solicitudes HTTP, se extraen las URL de las imágenes de los objetos de respuesta y se asignan a la variable imageURL1. Finalmente, se llama a la función setLoading con el argumento false para indicar que la carga de las imágenes ha finalizado y que la aplicación ya no está en estado de carga.

Para realizar la búsqueda de las ciudades se tiene el siguiente código, este bloque define una función asíncrona fetchFlightDestinations, que toma un parámetro

keyword que se utiliza para realizar la búsqueda de ciudades. Se utiliza la función fetch para realizar una solicitud HTTP GET a la API de Amadeus. La URL de la solicitud se construye dinámicamente utilizando la palabra clave proporcionada como parámetro. Se incluye un encabezado de autorización con un token de acceso para autenticarse con la API.

```
const fetchFlightDestinations = async (keyword) => {
  try {
    const response = await fetch(`https://test.api.amadeus.com/v1/reference-data/locations/cities?keyword=${keyword}`, {
      headers: {
        'Authorization': 'Bearer xLiDOID29OgR9qip0HOGiSesXJtw'
      }
    });
    if (!response.ok) {
      throw new Error('Error en la solicitud');
    }
    const data = await response.json();
    if (!data || !data.data || !Array.isArray(data.data)) {
      throw new Error('Data not found in response');
    }
    const destinationsWithIataCode = data.data.filter(destination => destination.iataCode);
    setFlightDestinations(destinationsWithIataCode);
    setLoading(false);
  } catch (error) {
    setLoading(false);
  }
};
```

Se utiliza el método json() para convertir la respuesta de la solicitud en un objeto JavaScript. Esto proporciona acceso a los datos devueltos por la API. Es importante filtrar los destinos de vuelo para asegurarse de que tengan un código IATA válido. Esto con el objetivo de que la búsqueda posterior de los vuelos sea exitosa.

Esta función se encarga de manejar la selección de una ciudad. Toma un objeto city como argumento que contiene la información de la ciudad, como el nombre y el código IATA. Luego, utiliza la función navigate del objeto navigation para navegar a la pantalla 'Home' y pasar la información de la ciudad seleccionada como un parámetro llamado selectedCity.

```
const handleFlightPress = (flight) => {
  setSelectedFlight(flight);
  setShowMasInfo(true);
};
```

Esta función se ejecuta cuando el usuario presiona una ciudad en la lista. Toma un objeto flight como argumento, que representa la ciudad seleccionada. Dentro de la función, se actualiza el estado selectedFlight con el valor seleccionado y se establece showMasInfo en true para mostrar más información sobre la ciudad.

```
const renderFlightItem = ({ item }) => (
  <TouchableOpacity onPress={() => handleFlightPress(item)}>
    <View style={styles.flightItem}>
      <Ionicons name="location-sharp" size={20} color="#0C3B2E" style={styles.icon} />
      <View>
        <Text style={styles.destination}>{item.name}</Text>
        <Text style={styles.details}>Tipo: {item.subType}</Text>
        <Text style={styles.details}>Código IATA: {item.iataCode}</Text>
        <Text style={styles.details}>País: {item.address.countryCode}</Text>
      </View>
    </View>
  </TouchableOpacity>
);
```

Esta función se utiliza para renderizar cada elemento de la lista de ciudades. Utiliza la función TouchableOpacity para crear un botón que el usuario puede presionar para seleccionar una ciudad. Cuando se presiona se llama a la función handleFlightPress con la ciudad como argumento. El contenido de cada elemento de la lista incluye el nombre del destino, el tipo de destino, el código IATA y el país del destino.

```
const handleSearch = () => {
  if (searchKeyword.trim() !== '') {
    fetchFlightDestinations(searchKeyword);
  }
};
```

Esta función se ejecuta cuando el usuario realiza una búsqueda. Comprueba si la palabra clave de búsqueda (searchKeyword) no está vacía y, si no lo está, llama a la función fetchFlightDestinations para buscar destinos basados en la palabra clave introducida por el usuario.

```
const handleCitySelect = (city) => {
  const { name, iataCode } = city;
  navigation.navigate('Home', { selectedCity: { name, iataCode } });
};
```

Se encarga de manejar la selección de una ciudad. Toma un objeto city como argumento que contiene la información de la ciudad, como el nombre y el código IATA. Luego, utiliza la función navigate del objeto navigation para navegar a la pantalla 'Home' y pasar la información de la ciudad seleccionada como un parámetro llamado selectedCity.

BookingScreen.js

Esta página se activa una vez que el usuario presiono el botón de buscar en la página anterior, aquí se manejan las especificaciones enviadas mediante parámetros por el usuario, haciendo la búsqueda mediante axios a la API de Amadeus.

```
import React, { useState } from 'react';
import { View, Text, Button } from 'react-native';

const HomeScreen = () => {
  const [hotels, setHotels] = useState([]);

  const searchHotels = async () => {
    try {
      const response = await fetch(`URL_DE_LA_API?habitaciones=${rooms}
      &destino=${selectedCity}&fecha_inicio=${startDate}&fecha_fin=${endDate}`);
      const data = await response.json();
      setHotels(data.hotels);
    } catch (error) {
      console.error('Error searching hotels:', error);
    }
  };

  return (
    <View>
      <Button title="Buscar Hoteles" onPress={searchHotels} />
      {hotels.map(hotel => (
        <View key={hotel.id}>
          <Text>{hotel.name}</Text>
          <Text>{hotel.location}</Text>
          <Text>{hotel.price}</Text>
        </View>
      ))}
    </View>
  );
};

export default HomeScreen;
```

const [hotels, setHotels] = useState([]): Define el estado hotels utilizando el hook useState. Inicialmente, se establece como un array vacío. Este estado se utilizará para almacenar los datos de los hoteles obtenidos de la API.

searchHotels: Se encarga de realizar la búsqueda de hoteles. Realiza una solicitud HTTP a una API utilizando los parámetros rooms, selectedCity, startDate y endDate. Luego, extrae los datos de la respuesta y actualiza el estado hotels con la lista de hoteles obtenida de la API. Si ocurre un error durante el proceso, se imprime un mensaje de error en la consola.

Button Buscar: Este es un botón que permite al usuario iniciar la búsqueda de hoteles. Cuando se presiona, se ejecuta la función searchHotels.

hotels.map: Este es un bloque de código que genera sobre la lista de hoteles almacenada en el estado hotels y renderiza la información de cada hotel en la interfaz de usuario.

Después cada hotel se representa como un componente View, y se le asigna una clave única id para identificarlo de manera eficiente durante el proceso de renderizado.

Login.js

En esta pantalla se lleva a cabo el inicio de sesión para usuarios que ya están registrados. Lo primero que se tiene dentro de la función de la pantalla es la variable que ayuda con la navegación, después se tiene la variable que guarda el estado del correo y la contraseña del usuario.

La función loginUser es una función asíncrona que se encarga de crear la conexión con Firebase para iniciar la sesión, recibe como parámetro el correo y la contraseña que el usuario proporciona. Dentro de la función se tiene un try-catch para manejar los posibles errores que se puedan generar con ayuda del await Firebase.auth() se llama a una función de Firebase para poder acceder a los correos guardados y una vez iniciada la sesión redirige a la pantalla de profileUser.

```
const Login = () => {
  const navigation = useNavigation();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  const loginUser = async (email, password) => {
    try {
      await firebase.auth().signInWithEmailAndPassword(email, password);
      navigation.navigate('ProfileUser');
    } catch (error) {
      alert(error.message);
    }
  };
```

```
const forgetPassword = () => {
  firebase.auth().sendPasswordResetEmail(email)
    .then(() => {
      alert('Se ha enviado un correo para restablecer la contraseña');
    }).catch((error) => {
      alert(error);
    });
};
```

Aquí se define una función llamada forgetPassword utilizando una función flecha que no recibe ningún parámetro. Dentro de la función, se llama a sendPasswordResetEmail del servicio de autenticación de Firebase. Esta función envía un correo electrónico para restablecer la contraseña a la dirección de correo electrónico proporcionada.

Para renderizar la pantalla del componente Login se utilizan componentes de la interfaz de usuario y algunas librerías para usar iconos de expo

El siguiente código define un componente para la interfaz de usuario del formulario

```
return (
  <View style={styles.container}>
    <Image source={require("../assets/logo.png")} style={styles.logo} />
    <View style={styles.formContainer}>
      <Text style={styles.title}>Iniciar Sesión</Text>
      <View style={styles.inputContainer}> ···
      </View>
      <View style={styles.inputContainer}>
        <Fontisto name="key" size={24} color="#6D9773" style={styles.icon} />
        <TextInput
          style={styles.input}
          placeholder="Password"
          onChangeText={(password) => setPassword(password)}
          autoCapitalize="none"
          autoCorrect={false}
          secureTextEntry={true}
          placeholderTextColor="#C0C0C0"
        />
      </View>
      <TouchableOpacity
        style={styles.button}
        onPress={() => loginUser(email, password)}
      >
        <View style={styles.buttonContent}>
          <Text style={styles.buttonText}>Entrar</Text>
          <FontAwesome
            name="sign-in"
            size={24}
            color="black"
            style={styles.iconText}
          />
        </View>
      </TouchableOpacity>
```

de inicio de sesión. Comienza con un contenedor view que dentro tiene otro view que agrupa el formulario, dentro contiene iconos y campos de entrada de texto para la contraseña. Este campo de entrada tiene atributos como placeholder, y onChangeText para manejar cambios de texto, desactivación de mayúsculas, autocorrección y entrada para contraseñas.

Finalmente, hay un botón que al presionarse llama a la función loginUser enviando los valores email y password.

Registration.js

Esta pantalla proporciona una interfaz para que un nuevo usuario se registre en la aplicación. Lo primero que se hace es importar la librería donde se tiene la configuración de Firebase, además de las librerías que se importan constantemente en las pantallas. Dentro de la función Registration se encuentra el hook de navegacion, y los estados de email, password, firstName y lastName que guardarán los datos ingresados por el usuario.

```
import React, { useState } from 'react';
import { View, Text, TouchableOpacity, TextInput, StyleSheet, Image, ScrollView, Alert } from 'react-native';
import { useNavigation } from '@react-navigation/native';
import { firebase } from '../config';
import { Fontisto } from '@expo/vector-icons';
import { FontAwesome } from '@expo/vector-icons';
const Registration = () => {
  const navigation = useNavigation();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [firstName, setFirstName] = useState('');
  const [lastName, setLastName] = useState('');
```

La función registerUser es una función asíncrona que dentro tiene un if que se encarga de verificar que los datos no estén vacíos, o de lo contrario lanza una alerta para avisar al usuario que debe llenar todos los campos. Por el contrario, cuando se llenan todos los campos se pasa a un bloque try-catch para tratar errores.

```
const registerUser = async () => {
  if (email.trim() === '' || password.trim() === '' || firstName.trim() === '' || lastName.trim() === '') {
    alert('Todos los campos son obligatorios');
    return;
  }
```

Dentro de este bloque, se utiliza un comando await que utiliza el método createUserWithEmailAndPassword de Firebase Auth para crear un nuevo usuario

con email y contraseña proporcionados mediante los parámetros. Este método es asincrónico y devuelve un resultado que se envía cuando el usuario ha sido creado. Si la creación del usuario falla se lanzará una excepción.

Después de que se crea el usuario, se obtiene el usuario que se acaba de crear con el currentUser, esta propiedad devuelve toda la información del usuario autenticado actualmente. Una vez que se obtiene el usuario actual, se envia un correo de verificación al correo proporcionado por el usuario y se verifica con el método sendEmailVerification.

```
try {
  await firebase.auth().createUserWithEmailAndPassword(email, password);
  const currentUser = firebase.auth().currentUser;

  await currentUser.sendEmailVerification({
    handleCodeInApp: true,
    url: "https://book-f12a2.firebaseapp.com",
  });
```

```
  await firebase.firestore().collection('users').doc(currentUser.uid).set({
    firstName,
    lastName,
    email,
  });
  Alert.alert(
    'Verifique su correo electrónico',
    'Para autenticar su cuenta',
    [
        {
            text: 'Aceptar',
            onPress: () => navigation.navigate('ProfileUser')
        }
    ]
);
```

En última instancia la información adicional que proporciono el usuario como el nombre, apellido y el email se guardan en una colección de Firestore llamada users y se crea una para cada usuario utilizando su UID como clave única para no confundir a los usuarios.

La continuación del código es la estructura del componente dentro de un scrollView para que la navegación sea más fácil, permitiendo desplazar verticalmente si el contenido no carga

```
return (
  <ScrollView contentContainerStyle={styles.container}>
    <Image source={require("../assets/logo.png")} style={styles.logo} />
    <View style={styles.formContainer}>
      <Text style={styles.title}>Registro</Text>
      <View style={styles.inputContainer}>
        <Fontisto
          name="person"
          size={24}
          color="#6D9773"
          style={styles.icon}
        />
        <TextInput
          style={styles.input}
          placeholder="Nombre"
          onChangeText={setFirstName}
          autoCorrect={false}
        />
      </View>
```

en la pantalla. Los campos de entrada para el nombre, apellido, email y contraseña están organizados en views individuales, cada uno con un icono y un TextInput para la entrada del usuario. Los campos de entrada incluyen propiedades para ajustar el comportamiento del teclado y la autocorrección. Por ‘ultimo el botón de registro ejecuta alguna acción cuando se presionar el botón registerUser.

```
      <View style={styles.inputContainer}> ···
      </View>
      <View style={styles.inputContainer}> ···
      </View>
      <View style={styles.inputContainer}> ···
      </View>
      <TouchableOpacity style={styles.button} onPress={registerUser}> ···
      </TouchableOpacity>

      <TouchableOpacity
        style={styles.registerLink}
        onPress={(() => navigation.navigate("Login"))}
      >
        <Text style={styles.registerText}>
          ¿Ya tienes una cuenta?{" "}
          <Text style={styles.registerTextHighlight}>Inicia sesión</Text>
        </Text>
      </TouchableOpacity>
    </View>
  </ScrollView>
);
```

Dashboard.js

Esta pantalla está diseñada para mostrar información acerca del usuario una vez que ya está autenticado y sus datos ya fueron recuperados de firestore. La librería nueva que se importa aquí es la del componente ImagePickerExample que se utiliza para seleccionar imágenes de la galería del usuario. Este componente utiliza dos hooks de estado, el de name que almacena el nombre de usuario y el loading que ayuda a manejar la carga de la pantalla mientras se cargan los datos de Firebase. El hook de useEffect se ejecuta cuando el componente se inicia y dentro de este se hace la recuperación de los datos del

```
import React, { useState, useEffect } from 'react';
import { SafeAreaView, Text, StyleSheet } from 'react-native';
import { firebase } from '../config';
import ImagePickerExample from './Image';
import { useNavigation } from '@react-navigation/native';

const Dashboard = () => {
    const navigation = useNavigation();
    const [name, setName] = useState('');
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        const user = firebase.auth().currentUser;
        if (user) {
            firebase.firestore().collection('users').doc(user.uid).get()
                .then((snapshot) => {
                    if (snapshot.exists) {
                        setName(snapshot.data().name);
                    } else {
                        console.log('Usuario no existe');
                    }
                    setLoading(false);
                }).catch((error) => {
                    console.error('Error obteniendo datos del usuario:', error);
                    setLoading(false);
                });
        } else {
            console.log('Usuario no autenticado');
            setLoading(false);
        }
    }, []);
```

usuario actual utilizando el Firebase.auth().currentUser. Si este usuario esta autentica entonces se intenta obtener los datos de la colección users utilizando su uid único. En caso de obtener un error entonces indica que la carga se terminó y no se recuperó ningún dato. Si loading cambia su estado a true entonces se cambia la vista y se muestra la interfaz inicial de la pantalla Dashboard, cargando dentro el componente de ImagePickerExample.

```
    if (loading) {
        return (
            <SafeAreaView style={styles.container}>
                <Text>Cargando...</Text>
            </SafeAreaView>
        );
    }

    return (
        <SafeAreaView style={styles.container}>
            <ImagePickerExample />
        </SafeAreaView>
    );
}

export default Dashboard;

const styles = StyleSheet.create({
    container: {
        flex: 1,
        alignItems: "center",
        justifyContent: "center",
    },
});
```

Image.js

```
export default function ImagePickerExample() {
    const [images, setImages] = useState([]);
    const navigation = useNavigation();

    const pickImage = async () => {
        let result = await ImagePicker.launchImageLibraryAsync({
            mediaTypes: ImagePicker.MediaTypeOptions.All,
            allowsEditing: true,
            aspect: [4, 3],
            quality: 1,
        });

        console.log(result);
```

Dentro de esta pantalla, se lleva a cabo toda la lógica de programación para acceder a la galería y que el usuario pueda elegir una fotografía para su perfil.

Primero se declara un estado images que almacenará la imagen seleccionada por el usuario. La función asíncrona pickImage se ejecuta para que el usuario pueda seleccionar una imagen de su biblioteca, la cual se abre mediante ImagePicker.launchImageLibraryAsync que a su vez tiene opciones específicas:

mediaTypes: Permite seleccionar cualquier tipo de medio (imágenes y videos).

allowsEditing: Si su estado se encuentra en true, permite la edición de la imagen seleccionada.

aspect: [4, 3]: Mantiene una relación de aspecto de 4:3 para la imagen seleccionada.

quality: 1: Establece la calidad de la imagen al máximo.

Después, dentro de la condición if se lleva a cabo la lógica para subir la imagen, primero verifica si el proceso de selección de imagen no fue cancelado por el usuario, después se obtiene la uri de la imagen seleccionada y con ayuda del fetch se obtiene los datos de la imagen. Seguido de eso con el blob se convierte la respuesta para que se conviertan en binarios los datos de la imagen. Para poder guardar la imagen y que sea visible solo para el usuario, se obtiene usuario ya autenticado y se define el nombre para guardar la imagen adjuntando el id del usuario. Después se crea una referencia para almacenar la imagen en el Firebase Storage utilizando el nombre definido anteriormente. Finalmente se sube el blob al almacenamiento del Firebase Storage y se actualiza el estado de images, mandando una alerta de confirmación o error en caso de presentar algún problema.

```
    if (!result.canceled) {
        try {
            const response = await fetch(result.assets[0].uri);
            const blob = await response.blob();
            const currentUser = firebase.auth().currentUser;
            const imageName = `${currentUser.uid}/profileImage`;
            const ref = storage.ref(imageName);
            await ref.put(blob);
            // Agregar la nueva imagen al estado
            setImages(prevImages => [...prevImages, { uri: result.assets[0].uri, name: imageName }]);

            // Mostrar alerta de éxito
            Alert.alert("Imagen Subida", "La imagen se ha subido correctamente.");
        } catch (error) {
            console.error("Error al subir la imagen:", error);
            // Mostrar alerta de error si la subida falla
            Alert.alert("Error", "Hubo un error al subir la imagen. Por favor, inténtalo de nuevo.");
        }
    }
};
```

Lo que se retorna dentro de la pantalla será el botón subir foto de perfil que llamará a la función pickImage y hace que la imagen se recupere una vez subida.

```
return (
    <View style={styles.container}>
        <TouchableOpacity onPress={pickImage} style={styles.profileButton}>
            {images.length > 0 ? (
                <Image source={{ uri: images[images.length - 1].uri }} style={styles.profileImage} />
            ) : (
                <Text style={styles.profileButtonText}>
                    Subir Foto de Perfil
                </Text>
            )}
        </TouchableOpacity>
    </View>
);
```

ProfileUser.js

Dentro de esta pantalla se realiza la recuperación de los datos para que el usuario pueda verlos recopilados. Lo primero que se tiene son los hooks de estado que guardaran los datos recuperados de Firebase.

```
const ProfileUser = () => {
    const navigation = useNavigation();
    const [email, setEmail] = useState('');
    const [firstName, setFirstName] = useState('');
    const [lastName, setLastName] = useState('');
    const [profileImage, setProfileImage] = useState(null);
    const [loading, setLoading] = useState(true);
```

Dentro de la función de loadData se hace la recuperación de los datos del usuario autenticado si el usuario no es nulo significa que se pueden cargan los datos satisfactoriamente.

```
const loadData = async () => {
    const currentUser = firebase.auth().currentUser;
    if (currentUser) {
        try {
            const url = await firebase.storage().ref(`${currentUser.uid}/profileImage`).getDownloadURL();
            setProfileImage(url);

            const userData = await firebase.firestore().collection('users').doc(currentUser.uid).get();
            if (userData.exists) {
                const userDataValues = userData.data();
                setFirstName(userDataValues.firstName);
                setLastName(userDataValues.lastName);
                setEmail(currentUser.email);
            }
            setLoading(false);
        } catch (error) {
            console.error('Error al cargar los datos:', error);
            setLoading(false);
        }
    }
};
```

Primero para poder recuperar la imagen del perfil, se accede al almacenamiento de Firebase Storage utilizando el uid del usuario autenticado y se obtiene la URL de descarga de la imagen de perfil almacenada en la ruta y se actualiza el estado de profileImage.

Ahora para poder recuperar los datos del usuario de Firebase, se accede a la colección users en Firestore y se obtiene el documento cuyo ID coincide con el uid del usuario autenticado. Para verificar si el documento existe se utiliza una condicional y en caso de existir se procede a extraer y utilizar los datos del documento con ayuda de usersData.data.

Para que los hooks de estado se actualicen con los datos correspondientes se hace uso del set y el nombre de cada estado. Finalmente, después de obtener todos los datos del usuario se establece el loading en false para indicar que la carga de datos se terminó y se renderiza todo el estilo de la pantalla con los datos existentes.

```
useEffect(() => {
    loadData();
}, []);

const signOut = async () => {
    try {
        await firebase.auth().signOut();
        Alert.alert(
            'Sesión cerrada',
            'Has cerrado sesión exitosamente.',
            [
                {
                    text: 'OK',
                    onPress: () => {
                        setEmail('');
                        setFirstName('');
                        setLastName('');
                        setProfileImage(null);
                        navigation.navigate('Lol');
                    }
                }
            ]
        );
    } catch (error) {
        console.error('Error al cerrar sesión:', error);
        Alert.alert('Error', 'Hubo un error al cerrar sesión.
    }
};
```

useEffect es un hook que permite ejecutar efectos de carga en los componentes, dentro de este se llama a la función loadData que es una función asíncrona que carga los datos del usuario actual desde Firebase, después en la función signOut se ejecuta cuando el usuario cierre la sesión y limpia los estados de navegación y donde se guardan los datos del usuario.

La función change password es para que el usuario pueda cambiar si contraseña, para esto la función recopila el correo del usuario y le envía un correo al usuario para autenticar que es él y desde ese correo pueda cambiar su contraseña. Si la promesa que envía el correo se vuelve exitosa, se ejecuta el bloque que envia una alerta para que el usuario debe revisar su correo electrónico para continuar su proceso.

```
const changePassword = () => {
    const currentUser = firebase.auth().currentUser;
    if (currentUser) {
        firebase.auth().sendPasswordResetEmail(currentUser.email)
            .then(() => {
                alert('Favor de revisar su correo electrónico');
            }).catch((error) => {
                alert(error);
            });
    } else {
        alert('No se pudo enviar el correo de restablecimiento de contraseña.');
    }
};
```

```
const takePhoto = () => {
    console.log("Navigating to TakePhoto");
    navigation.navigate("Tomar");
};
```

La función takePhoto se usa para llamar al componente Tomar que abre una pantalla para que el usuario pueda tomar una fotografía desde su cámara del dispositivo.

Ahora bien, cuando el usuario quiere cambiar su información de perfil como el nombre o el apellido, se tiene la función updateProfile que actualiza el perfil del usuario en Firebase. Primero, se obtiene que el usuario este autenticado y se actualiza la información de users, si el proceso es satisfactorio envía una alerta de éxito, de lo contrario envía una alerta de error.

```
const updateProfile = async () => {
    const currentUser = firebase.auth().currentUser;
    if (currentUser) {
        try {
            await firebase.firestore().collection('users').doc(currentUser.uid).update({
                firstName: firstName,
                lastName: lastName,
            });
            Alert.alert('Perfil Actualizado', 'Tu información ha sido actualizada correctamente.');
        } catch (error) {
            console.error('Error al actualizar el perfil:', error);
            Alert.alert('Error', 'Hubo un error al actualizar tu perfil. Por favor, inténtalo de nuevo.');
        }
    } else {
        Alert.alert('Error', 'No se pudo actualizar el perfil. Usuario no autenticado.');
    }
};
```

El código crea una pantalla de perfil de usuario con ayuda de un ScrollView para permitir el desplazamiento. Dentro, hay varios campos de texto para editar el

nombre y apellido, pero el correo electrónico no es editable. Hay botones para cambiar la contraseña, cerrar sesión, cambiar la foto de perfil, guardar cambios y tomar una foto. Si hay una imagen de perfil, se muestra; si no, se muestra un componente Dashboard.

```
<SafeAreaView style={styles.container}>
    <ScrollView contentContainerStyle={styles.scrollViewContainer}>
        <View style={styles.formContainer}>
            <TextInput
                style={styles.TextInput}
                value={firstName}
                onChangeText={setFirstName}
                placeholder="Nombre"
            />
            <TextInput
                style={styles.TextInput}
                value={lastName}
                onChangeText={setLastName}
                placeholder="Apellido"
            />
            <TextInput
                style={styles.TextInput}···
                editable={false}
            />
            <TouchableOpacity···
            </TouchableOpacity>
            <TouchableOpacity···
            </TouchableOpacity>
            <TouchableOpacity
                onPress={pickImage}
                style={styles.button}>
                <Text style={{ fontWeight: "bold", fontSize: 22 }}>Cambiar Foto de Perfil</Text>
            </TouchableOpacity>
            {profileImage ? (
                <Image source={{ uri: profileImage }} style={styles.profileImage} />
            ) : (
                <Dashboard />
            )}
            <TouchableOpacity
                onPress={updateProfile}
                style={styles.button}
            >
                <Text style={{ fontWeight: "bold", fontSize: 22 }}>Guardar Cambios</Text>
            </TouchableOpacity>
            <TouchableOpacity
                onPress={takePhoto}
                style={styles.button}
            >
                <Text style={{ fontWeight: "bold", fontSize: 22 }}>Tomar Foto</Text>
            </TouchableOpacity>
```

Ilustración 13 Pantalla Dashboard

TakePhoto.js

Esta pantalla se activa cuando se presiona el botón tomar foto, se abre la cámara del usuario. Primero se importan varias librerías y componentes como CameraView y useCameraPermissions de "expo-camera" para gestionar la cámara y sus permisos, igualmente MediaLibrary de "expo-media-library" para acceder a la galería de medios, y configuración de Firebase desde un archivo local.

```
import { CameraView, useCameraPermissions } from "expo-camera";
import * as MediaLibrary from "expo-media-library";
import { firebase, storage } from "../config";
import { useNavigation } from "@react-navigation/native";
import { Entypo } from "@expo/vector-icons";
import { FontAwesome6 } from "@expo/vector-icons";
import { MaterialIcons } from '@expo/vector-icons';
import { FontAwesome } from '@expo/vector-icons';
```

Lol.js

Esta es la primera pantalla que se ve cuando se inicia la aplicación por primera vez, aparece una imagen que busca resumir la función de la aplicación, también aparece una leyenda y un botón que indica al usuario que comience su experiencia. Se utiliza un useEffect estado que ayuda a crear una animación para cuando se inicia la pantalla y que se aparezca con un deslizamiento de izquierda a derecha.

```
import React, { useState, useRef, useEffect } from 'react';
import { View, Text, TouchableOpacity, StyleSheet, Animated, Image } from 'react-native';
import { useNavigation } from '@react-navigation/native';

const BookingScreen = () => {
  const navigation = useNavigation();
  const slideAnim = useRef(new Animated.Value(-200)).current;

  useEffect(() => {
    Animated.timing(slideAnim, {
      toValue: 0,
      duration: 2000,
      useNativeDriver: true,
    }).start();
  }, [slideAnim]);

  const goToLogin = () => {
    navigation.navigate('Login');
  };

  return (
    <View style={styles.container}>
      <Animated.View style={[styles.imageContainer, { transform: [{ translateX: slideAnim }] }]}>
        <Image
          source={require('../assets/lol.png')}
          style={styles.image}
        />
      </Animated.View>
      <Animated.View style={[styles.textContainer, { transform: [{ translateX: slideAnim }] }]}>

        <Text style={styles.welcomeText}>Bienvenido a Booking</Text>

        <Text style={styles.subtitleText}>Reserva tu próximo vuelo, hotel o automóvil con nosotros</Text>
        <TouchableOpacity style={styles.button} onPress={goToLogin}>
          <Text style={styles.buttonText}>Comenzar</Text>
        </TouchableOpacity>
      </Animated.View>
    </View>
  );
```

ResultScreen.js

La pantalla resultados se utilizan varios hooks para obtener y mostrar una lista de hoteles de acuerdo a la ciudad especificada, para la búsqueda de hoteles intenta obtener los hoteles desde Firebase y si no encuentra ninguno, consulta una API externa para buscar hoteles en la ciudad y guarda los datos obtenidos en Firestore. Los hoteles se pueden filtrar por precio o nombre mediante un modal que permite al usuario aplicar estos filtros. La pantalla muestra una lista de hoteles con detalles como imagen, nombre, precio, estrellas y ubicación, y permite al usuario buscar hoteles por nombre. Si se selecciona un hotel, se navega a una pantalla de reserva con los detalles del hotel seleccionado y las fechas de estadía. Además, la pantalla muestra un indicador de carga mientras se obtienen los datos.

```
const fetchHotels = async (cityName) => {
  try {
    const existingHotels = await getHotelsByCityFromFirestore(cityName);
    if (existingHotels.length > 0) {
      setHotels(existingHotels);
      sortHotels(sortOrder, existingHotels);
    } else {
      const params = {
        name: cityName,
        featureCode: "HTL",
        username: "ferchis26",
      };
      const url = `http://api.geonames.org/searchJSON?${new URLSearchParams(
        params
      ).toString()}`;
      const response = await axios.get(url);
      const data = response.data;
      if (!data || !data.geonames || data.geonames.length === 0) {
        throw new Error(
          "No se encontraron hoteles en la ciudad especificada."
        );
      }

      const limitedData = data.geonames.slice(0, 5);

      const hotelsData = await Promise.all(
        limitedData.map(async (hotel) => {
          const imageUrl = await fetchHotelImage(hotel.name);

          // Generar un número aleatorio entre 1 y el número total de especificaciones disponibles
          const numSpecs = Math.floor(Math.random() * specifications.length) + 1;
          const selectedSpecs = generateRandomSpecifications(numSpecs);

          const hotelData = {
            id: hotel.geonameId,
            name: hotel.name,
            description: generateRandomDescription(),
            imageUrl,
            stars: Math.floor(Math.random() * 5) + 1,
            pricePerNight: (Math.random() * 200 + 50).toFixed(2),
            location: `${hotel.name}, ${hotel.countryName}`,
            availableRooms: {
              individual: Math.floor(Math.random() * (200 - 60 + 1)) + 60,
              shared: Math.floor(Math.random() * (170 - 40 + 1)) + 40,
            },
            specifications: selectedSpecs,
          };

          await saveHotelToFirestore(hotelData);

          return hotelData;
        })
      );

      function generateRandomSpecifications(numSpecs) {
        const randomSpecs = [];
        const availableSpecs = [...specifications]; // Copiar la lista de especificaciones disponibles

        // Escoger numSpecs especificaciones aleatorias
        for (let i = 0; i < numSpecs; i++) {
          const randomIndex = Math.floor(Math.random() * availableSpecs.length);
          const spec = availableSpecs[randomIndex];
          randomSpecs.push(spec);
          availableSpecs.splice(randomIndex, 1); // Eliminar la especificación seleccionada para no repetirla
        }

        return randomSpecs;
      }
```

```
      setHotels(hotelsData);
      sortHotels(sortOrder, hotelsData);
    }
  } catch (error) {
  } finally {
    setLoading(false);
  }
};
const specifications = [
  { id: 1, name: 'Internet', icon:'wifi' },
  { id: 2, name: 'Restaurante',icon:'rest' },
  { id: 3, name: 'Lavandería', icon:'checkcircle' },
  { id: 4, name: 'Estacionamiento',icon:'car' },
];

const fetchHotelImage = async (hotelName) => {
  try {
    const response = await axios.get(
      "https://api.unsplash.com/photos/random",
      {
        params: {
          query: `hotel`,
          orientation: "landscape",
          client_id: "ypGbeZFXaNXsiHP77Wk41gcerLfkx1BnsnDBi76vVws",
        },
      }
    );
    return response.data.urls.regular;
  } catch (error) {
    return "https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcR4CTQo_YrIea8Dfle4Jjh4Hf9aYTGxNePgCw&s";
  }
};
```

```
const saveHotelToFirestore = async (hotelData) => {
  try {
    const db = firebase.firestore();
    const existingHotel = await db
      .collection("hotelsPreference")
      .doc(hotelData.id.toString())
      .get();
    if (!existingHotel.exists) {
      await db
        .collection("hotelsPreference")
        .doc(hotelData.id.toString())
        .set(hotelData);
      console.log(`Hotel ${hotelData.name} guardado en Firestore`);
    } else {
      console.log(`Hotel ${hotelData.name} ya existe en Firestore`);
    }
  } catch (error) {
    console.error(
      `Error al guardar hotel ${hotelData.name} en Firestore:`,
      error
    );
  }
};
```

```
const getHotelsByCityFromFirestore = async (cityName) => {
  try {
    const db = firebase.firestore();
    const snapshot = await db.collection("hotelsPreference").get();
    const hotels = [];
    snapshot.forEach((doc) => {
      const data = doc.data();
      if (data.location.includes(cityName)) {
        hotels.push(data);
      }
    });
    return hotels;
  } catch (error) {
    console.error(
      `Error al obtener hoteles de la ciudad ${cityName} de Firestore:`,
      error
    );
    return [];
  }
};

const generateRandomDescription = () => {
  const descriptions = [
    "Un lugar ideal para tus vacaciones.",
    "Experimenta la comodidad y el lujo en su máxima expresión.",
    "Descubre la hospitalidad y el encanto de este hotel.",
    "Perfecto para escapadas románticas y familiares por igual.",
    "Disfruta de vistas impresionantes y servicios de primera clase.",
  ];
  const randomIndex = Math.floor(Math.random() * descriptions.length);
  return descriptions[randomIndex];
};
```

```
const handleReservePress = (hotel) => {
  navigation.navigate("Reservar", {
    hotel,
    rooms: rooms,
    startDate: startDate,
    endDate: endDate,
  });
};
```

```
const openFiltersModal = () => {
  setShowFiltersModal(true);
};

const closeFiltersModal = () => {
  setShowFiltersModal(false);
};

const capitalizeFirstLetter = (input) => {
  if (input.length === 0) return input;
  return input.charAt(0).toUpperCase() + input.slice(1);
};

const sortHotels = (orderBy, order, hotelsToSort) => {
  let sorted = [...hotelsToSort];
  if (orderBy === "price") {
    sorted = sorted.sort((a, b) => {
      if (order === "asc") {
        return parseFloat(a.pricePerNight) - parseFloat(b.pricePerNight);
      } else {
        return parseFloat(b.pricePerNight) - parseFloat(a.pricePerNight);
      }
    });
  } else if (orderBy === "name") {
    sorted = sorted.sort((a, b) => {
      if (order === "asc") {
        return a.name.localeCompare(b.name); // Orden ascendente por nombre
      } else {
        return b.name.localeCompare(a.name); // Orden descendente por nombre
      }
    });
  }
  setSortedHotels(sorted);
};
```

```
const applyFilters = () => {
  if (activeFilter === "name") {
    sortHotels("name", sortByNameOrder, hotels);
  } else {
    sortHotels("price", sortOrder, hotels);
  }
  closeFiltersModal();
};

if (loading) {
  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <ActivityIndicator size="large" color="#0000ff" />
      <Text>Cargando...</Text>
    </View>
  );
}
```

Calificar.js

Dentro de la pantalla se permite a los usuarios calificar y dejar opiniones sobre un hotel específico. Utiliza Firebase para autenticar al usuario y manejar la base de datos. Al cargar el componente, se configuran los encabezados de navegación y se obtiene el ID del usuario autenticado. Luego, si se proporcionan datos del hotel, se obtienen y muestran las opiniones existentes. Los usuarios pueden escribir una opinión, asignar una calificación mediante estrellas y enviar su reseña. Si ya existe una opinión del usuario para el hotel, se le da la opción de actualizarla. Las opiniones de otros usuarios se muestran en una lista, incluyendo sus nombres y calificaciones. El diseño incluye varios componentes de estilo, como imágenes, texto y botones estilizados.

```javascript
useEffect(() => {
  configureHeader();
  if (hotelData) {
    setHotel(hotelData);
  }
  fetchUserId();
}, [hotelData]);

useEffect(() => {
  if (hotel) {
    fetchOpiniones();
  }
}, [hotel]);

const fetchUserId = async () => {
  const user = firebase.auth().currentUser;
  if (user) {
    setUserId(user.uid);
    setUserName(user.displayName || "Nombre de usuario");
  } else {
    console.error("No hay un usuario autenticado");
  }
};
```

```javascript
const fetchOpiniones = async () => {
  const db = firebase.firestore();
  try {
    const opinionesSnapshot = await db
      .collection("opiniones")
      .where("hotelId", "==", hotel.id)
      .get();

    const opinionesData = opinionesSnapshot.docs.map((doc) => ({
      id: doc.id,
      ...doc.data(),
    }));

    await Promise.all(
      opinionesData.map(async (opinion) => {
        if (opinion.userId) {
          const userDoc = await db.collection("users").doc(opinion.userId).get();
          if (userDoc.exists) {
            opinion.userName = userDoc.data().displayName || "Anónimo";
          } else {
            opinion.userName = "Anónimo";
          }
        } else {
          opinion.userName = "Anónimo";
        }
      })
    );

    setOpiniones(opinionesData);
    setLoadingOpiniones(false);
  } catch (error) {
    console.error("Error al obtener opiniones:", error);
    setLoadingOpiniones(false);
  }
};
```

```
const db = firebase.firestore();
try {
  const opinionQuery = await db
    .collection("opiniones")
    .where("userId", "==", userId)
    .where("hotelId", "==", hotel.id)
    .get();

  if (!opinionQuery.empty) {
    Alert.alert(
      "Ya has puntuado este hotel",
      "¿Deseas cambiar tu opinión?",
      [
        {
          text: "Cancelar",
          style: "cancel",
        },
        {
          text: "Aceptar",
          onPress: async () => {
            const existingOpinionId = opinionQuery.docs[0].id;
            await db
              .collection("opiniones")
              .doc(existingOpinionId)
              .update({
                comment: comment,
                rating: rating,
              });
            console.log("Opinión actualizada con éxito");
            navigation.goBack();
          },
        },
      ]
    );
  } else {
    await db.collection("opiniones").add({
      userId: userId,
      hotelId: hotel.id,
      comment: comment,
      rating: rating,
      createdAt: firebase.firestore.FieldValue.serverTimestamp(),
    });
    console.log("Nueva opinión guardada con éxito");
    navigation.goBack();
  }
} catch (error) {
  console.error("Error al guardar o actualizar la opinión:", error);
}
};

const renderStars = () => {
  const maxStars = 5;
  let stars = [];
  for (let i = 1; i <= maxStars; i++) {
    stars.push(
      <TouchableOpacity key={i} onPress={() => handleRating(i)}>
        <AntDesign
          name={i <= rating ? "star" : "staro"}
          size={30}
          color={i <= rating ? "#FFD700" : "#ccc"}
          style={{ marginRight: 5 }}
        />
      </TouchableOpacity>
    );
  }
  return stars;
};
```

```
const renderStars2 = (nom) => {
  const maxStars = 5;
  let stars = [];
  for (let i = 1; i <= maxStars; i++) {
    stars.push(
      <AntDesign
        key={i}
        name={i <= nom ? "star" : "staro"}
        size={20}
        color={i <= nom ? "#FFD700" : "#ccc"}
        style={{ marginRight: 5 }}
      />
    );
  }
  return stars;
};


const handleRating = (ratingValue) => {
  setRating(ratingValue);
};

if (!hotel) {
  return <Text>Cargando...</Text>;
}
```

Favs.js

El componente permite a los usuarios ver una lista de sus aeropuertos favoritos. Utiliza estados para almacenar los IDs y datos de los aeropuertos favoritos, respectivamente. Al ejecutar el componente, se lama a fetchFavorites que obtiene los IDs de los aeropuertos favoritos del usuario autenticado desde Firebase Firestore. Luego, se llama a fetchFavoriteAirports con esos IDs para obtener los datos completos de los aeropuertos favoritos, los cuales se almacenan en el estado favoriteAirports. La función renderItem se encarga de mostrar cada aeropuerto favorito en un diseño que incluye su imagen, nombre, descripción y ubicación. Finalmente, FlatList se usa para renderizar la lista de aeropuertos favoritos de manera eficiente.

```
const fetchFavorites = async () => {
  try {
    const db = firebase.firestore();
    const uid = firebase.auth().currentUser.uid;
    const favoritesRef = db.collection('favorites').doc(uid);
    const doc = await favoritesRef.get();
    if (doc.exists) {
      setFavorites(doc.data().favorites);
      fetchFavoriteAirports(doc.data().favorites);
    } else {
      console.log('No hay datos de favoritos para este usuario.');
    }
  } catch (error) {
    console.error('Error al obtener favoritos:', error);
  }
};
```

```
const fetchFavoriteAirports = async (favoriteIds) => {
  try {
    const db = firebase.firestore();
    const favoriteAirports = [];

    await Promise.all(
      favoriteIds.map(async (id) => {
        const doc = await db.collection('airportsPreference').doc(id.toString()).get();
        if (doc.exists) {
          favoriteAirports.push(doc.data());
        }
      })
    );

    setFavoriteAirports(favoriteAirports);
  } catch (error) {
    console.error('Error al obtener aeropuertos favoritos:', error);
  }
};
```

```
const renderItem = ({ item }) => {
  return (
    <View style={styles.itemContainer}>
      <Image source={{ uri: item.imageUrl }} style={styles.itemImage} />
      <View style={styles.itemTextContainer}>
        <Text style={styles.itemTitle}>{item.name}</Text>
        <Text style={styles.itemSubtitle}>{item.description}</Text>
        <Text style={styles.itemLocation}>{item.location}</Text>
      </View>
    </View>
  );
};
```

HotelesSearch.js

El componente permite a los usuarios buscar, filtrar y clasificar hoteles desde una base de datos Firestore. Al ejecutar, se obtiene la lista de hoteles y se configura la navegación. Los usuarios pueden buscar hoteles por nombre, ordenar la lista por precio o nombre y aplicar filtros mediante un modal. Los datos de los hoteles se muestran en una lista con imagen, nombre, precio y ubicación, y cada hotel tiene un botón para calificar. La interfaz se compone de varios elementos estilizados, incluyendo botones y entradas de texto para buscar y filtrar hoteles.

```
const fetchHotels = async () => {
  try {
    setLoading(true);
    const existingHotels = await getHotelsFromFirestore();
    setHotels(existingHotels);
    sortHotels(sortOrder, existingHotels);
    setLoading(false);
  } catch (error) {
    console.error("Error al buscar hoteles:", error);
    setLoading(false);
  }
};

const saveHotelToFirestore = async (hotelData) => {
  try {
    const db = firebase.firestore();
    await db
      .collection("hotelsPreference")
      .doc(hotelData.id.toString())
      .set(hotelData);
    console.log(`Hotel ${hotelData.name} guardado en Firestore`);
  } catch (error) {
    console.error(
      `Error al guardar hotel ${hotelData.name} en Firestore:`,
      error
    );
  }
};
```

```
const getHotelsFromFirestore = async () => {
  try {
    const db = firebase.firestore();
    const snapshot = await db.collection("hotelsPreference").get();
    const hotels = [];
    snapshot.forEach((doc) => {
      const data = doc.data();
      hotels.push(data);
    });
    return hotels;
  } catch (error) {
    console.error("Error al obtener hoteles de Firestore:", error);
    return [];
  }
};

const handleReservePress = (hotel) => {
  navigation.navigate("Calificar", {hotel});
};

const configureHeader = () => {…
};

const openFiltersModal = () => {
  setShowFiltersModal(true);
};

const closeFiltersModal = () => {
  setShowFiltersModal(false);
};
```

```
const sortHotels = (orderBy, hotelsToSort) => {
  let sorted = [...hotelsToSort];
  if (orderBy === "price") {
    sorted = sorted.sort((a, b) => {
      if (sortOrder === "asc") {
        return parseFloat(a.pricePerNight) - parseFloat(b.pricePerNight);
      } else {
        return parseFloat(b.pricePerNight) - parseFloat(a.pricePerNight);
      }
    });
  } else if (orderBy === "name") {
    sorted = sorted.sort((a, b) => {
      if (sortByNameOrder === "asc") {
        return a.name.localeCompare(b.name);
      } else {
        return b.name.localeCompare(a.name);
      }
    });
  }
  setSortedHotels(sorted);
};

const applyFilters = () => {
  if (activeFilter === "name") {
    sortHotels("name", hotels);
  } else {
    sortHotels("price", hotels);
  }
  closeFiltersModal();
};

const onEndReached = () => {
};
```

```
const applyFilters = () => {
  if (activeFilter === "name") {
    sortHotels("name", hotels);
  } else {
    sortHotels("price", hotels);
  }
  closeFiltersModal();
};

const onEndReached = () => {
};

const capitalizeFirstLetter = (input) => {
  if (input.length === 0) return input;
  return input.charAt(0).toUpperCase() + input.slice(1);
};

const searchHotels = (text) => {
  const searchText = capitalizeFirstLetter(text.trim());
  const filteredHotels = hotels.filter((hotel) =>
    hotel.name.toLowerCase().includes(searchText.toLowerCase())
  );
  setSortedHotels(filteredHotels);
};
```

Reservar.js

El componente permite a los usuarios reservar habitaciones en un hotel seleccionado, mostrando detalles del hotel, incluyendo su imagen, nombre, estrellas, precio, ubicación, descripción y especificaciones. Los usuarios pueden seleccionar fechas de inicio y fin, el número de habitaciones y el tipo de habitación (individual o compartida). La reserva se guarda en Firestore, y se muestra un mensaje de éxito o error. La interfaz incluye un encabezado configurado con un

título personalizado y una barra de navegación estilizada, además de elementos como campos de texto y botones de opción para facilitar la interacción del usuario.

```
useEffect(() => {
  configureHeader();
}, []);

const handleReserve = async () => {
  if (!startDate || !endDate) {
    Alert.alert("Error", "Por favor selecciona fechas válidas para la reserva.");
    return;
  }

  try {
    const reservationData = {
      hotelId: hotel.id,
      hotelName: hotel.name,
      startDate,
      endDate,
      roomType: selectedRoomType,
    };
    const db = firebase.firestore();
    await db.collection("reservations").add(reservationData);
    Alert.alert("Éxito", "¡Reserva realizada con éxito!");
    navigation.goBack();
  } catch (error) {
    console.error("Error al realizar la reserva:", error);
    Alert.alert("Error", "Ocurrió un error al realizar la reserva. Por favor intenta de nuevo más tarde.");
  }
};
```

Vuelos.js

El componente Vuelos permite a los usuarios buscar aeropuertos en una ciudad ingresada, mostrando información detallada y ofreciendo opciones de filtrado por precio y nombre. Utiliza Firebase para almacenar y recuperar datos de aeropuertos y favoritos del usuario, y se conecta a la API de Geonames para obtener información de aeropuertos. También obtiene imágenes aleatorias de Unsplash para los aeropuertos. El componente presenta una interfaz con un campo de entrada para la ciudad, un botón de filtros que abre un modal para seleccionar los criterios de orden, y una lista de resultados que incluye el nombre, la descripción, la ubicación y una opción para marcar como favorito.

```
useEffect(() => {
  fetchAirports(capitalizeFirstLetter(cityName));
  configureHeader();
}, [cityName]);

useEffect(() => {
  fetchFavorites();
}, []);

useEffect(() => {
  fetchRandomImages();
}, [airports]);

const fetchAirports = async (cityName) => {
  try {
    const existingAirports = await getAirportsByCityFromFirestore(cityName);
    if (existingAirports.length > 0) {
      setAirports(existingAirports);
      sortAirports(sortOrder, existingAirports);
    } else {
      const params = {
        q: cityName,
        featureCode: "AIRP",
        username: "ferchis26",
      };
      const url = `http://api.geonames.org/searchJSON?${new URLSearchParams(
        params
      ).toString()}`;
      const response = await axios.get(url);
      const data = response.data;
      if (!data || !data.geonames || data.geonames.length === 0) {
        throw new Error(
          "No se encontraron aeropuertos en la ciudad especificada."
        );
      }
      const limitedData = data.geonames.slice(0, 5);

      const airportsData = await Promise.all(
        limitedData.map(async (airport) => {
          const airportData = {
            id: airport.geonameId,
            name: airport.name,
            description: generateRandomAirportDescription(),
            location: `${airport.name}, ${airport.countryName}`,
          };

          await saveAirportToFirestore(airportData);

          return airportData;
        })
      );

      setAirports(airportsData);
      sortAirports(sortOrder, airportsData);
    }
  } catch (error) {
    console.error("Error al buscar aeropuertos:", error);
  } finally {
    setLoading(false);
  }
};
```

```
const getAirportsByCityFromFirestore = async (cityName) => {
  try {
    const db = firebase.firestore();
    const snapshot = await db.collection("airportsPreference").get();
    const airports = [];
    snapshot.forEach((doc) => {
      const data = doc.data();
      if (data.location.includes(cityName)) {
        airports.push(data);
      }
    });
    return airports;
  } catch (error) {
    console.error(
      `Error al obtener aeropuertos de la ciudad ${cityName} de Firestore:`,
      error
    );
    return [];
  }
};

const saveAirportToFirestore = async (airportData) => {
  try {
    const db = firebase.firestore();
    const existingAirport = await db
      .collection("airportsPreference")
      .doc(airportData.id.toString())
      .get();
    if (!existingAirport.exists) {
      airportData.imageUrl = await fetchHotelImage(airportData.name);
      await db
        .collection("airportsPreference")
        .doc(airportData.id.toString())
        .set(airportData);
      console.log(`Aeropuerto ${airportData.name} guardado en Firestore`);
    } else {
      console.log(`Aeropuerto ${airportData.name} ya existe en Firestore`);
```

```
const fetchFavorites = async () => {
  try {
    const db = firebase.firestore();
    const uid = firebase.auth().currentUser.uid;
    const favoritesRef = db.collection("favorites").doc(uid);
    const doc = await favoritesRef.get();
    if (doc.exists) {
      setFavorites(doc.data().favorites);
    } else {
      console.log("No hay datos de favoritos para este usuario.");
    }
  } catch (error) {
    console.error("Error al obtener favoritos:", error);
  }
};

const toggleFavorite = async (airportId) => {
  try {
    const db = firebase.firestore();
    const uid = firebase.auth().currentUser.uid;
    const favoritesRef = db.collection("favorites").doc(uid);

    let updatedFavorites = [...favorites];

    if (updatedFavorites.includes(airportId)) {
      updatedFavorites = updatedFavorites.filter((id) => id !== airportId);
    } else {
      updatedFavorites.push(airportId);
    }

    await favoritesRef.set({ favorites: updatedFavorites });

    setFavorites(updatedFavorites);
  } catch (error) {
    console.error("Error al actualizar favoritos:", error);
  }
```

VerReserva.js

El componente VerReserva gestiona la visualización de reservas de usuario desde una base de datos Firestore. Al inicio, configura el estilo del encabezado de navegación y recupera las reservas del usuario actual mediante una consulta que filtra por su ID. Utiliza useEffect para ejecutar esta consulta al cargar el componente

y al actualizarlo, asegurando así que siempre muestra datos actualizados. Cada reserva se muestra en una lista con detalles como nombre del hotel, fechas de inicio y fin, tipo de habitación y estado actual. Además, permite cancelar reservas individualmente, actualizando la lista de reservas mostrada y eliminando la reserva correspondiente en Firestore. Si no hay reservas, se muestra un mensaje indicando la falta de ellas.

```
const VerReserva = () => {
  const configureHeader = () => {
    navigation.setOptions({
      headerShown: true,
      title: "Booking",
      headerTitleStyle: {
        fontSize: 25,
        fontWeight: "bold",
        color: "white",
        textAlign: "center",
      },
      headerLeft: null,
      headerStyle: {
        backgroundColor: "#0C3B2E",
        height: 120,
        borderBottomColor: "transparent",
        shadowColor: "transparent",
      },
      headerRight: () => (
        <Entypo
          name="emoji-happy"
          size={24}
          color="#FFBA00"
          style={{ marginRight: 12 }}
        />
      ),
    });
  };
  const [reservations, setReservations] = useState([]);
  const navigation = useNavigation();
```

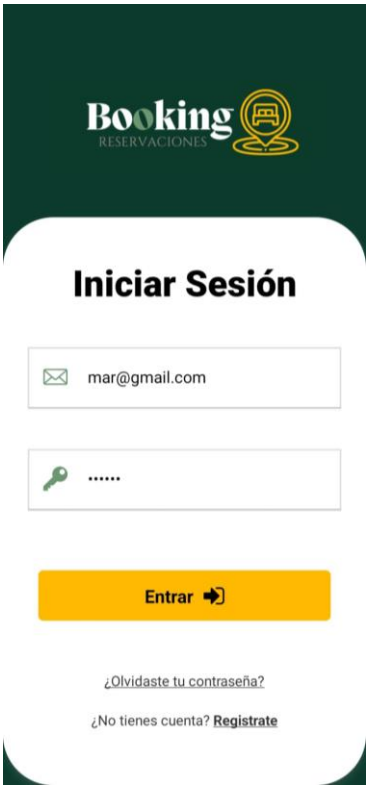
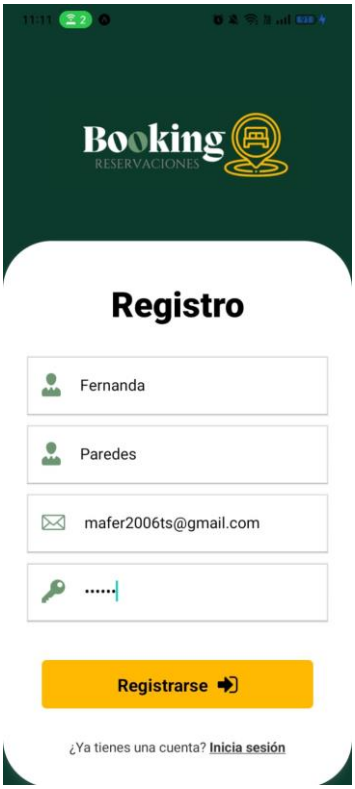
```
useEffect(() => {
  configureHeader();
  const fetchReservations = async () => {
    try {
      const currentUser = firebase.auth().currentUser;
      if (!currentUser) {
        console.log("Usuario no autenticado.");
        return;
      }

      const db = firebase.firestore();
      const snapshot = await db
        .collection("reservations")
        .where("userId", "==", currentUser.uid)
        .get();

      const reservationsData = [];
      const currentDate = new Date();

      snapshot.forEach((doc) => {
        const data = doc.data();
        const endDate = new Date(data.endDate);
        if (currentDate > endDate) {
          data.status = "vencido";
        }
        reservationsData.push({ id: doc.id, ...data });
      });

      setReservations(reservationsData);
    } catch (error) {
      console.error("Error al obtener las reservas:", error);
    }
  };

  fetchReservations();
}, []);
```

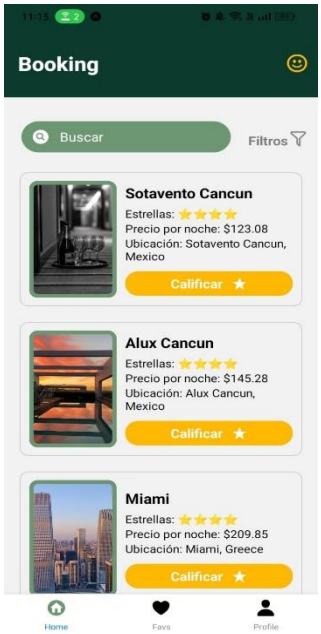
```
const handleCancelReservation = async (reservationId) => {
  try {
    const db = firebase.firestore();
    await db.collection("reservations").doc(reservationId).delete();
    setReservations((prevReservations) =>
      prevReservations.filter(
        (reservation) => reservation.id !== reservationId
      )
    );
    console.log("Reservación cancelada y eliminada.");
  } catch (error) {
    console.error("Error al cancelar la reservación:", error);
  }
};
```

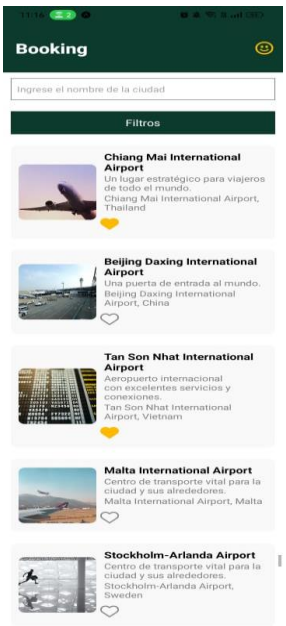
Este fue el código de todas las pantallas que se utilizan para la ejecución de aplicación móvil.

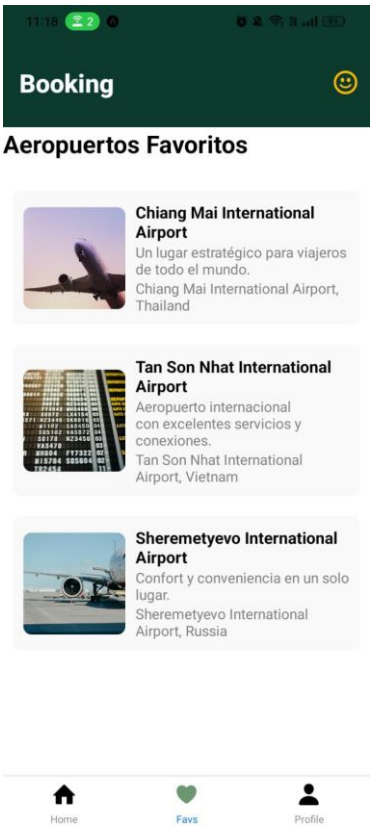
Ejecución

La ejecución final de la aplicación se muestra en el siguiente apartado, dentro de este se pueden visualizar las pantallas finales ejecutándose desde un teléfono celular.

Loading Page: Se inicia cuando carga por primera vez la app.	Login: Página para iniciar sesión.	Registro: Página para registrar un nuevo usuario.
		

Pantalla de inicio para elegir su destino.	Pantalla para buscar ciudades.	Página para buscar hoteles y puntuarlos.
		

Pantalla para agregar una calificación y comentario a los hoteles.	Busca los aeropuertos disponibles.	Muestra las reservas de los usuarios y sus datos.
		

Muestra los hoteles favoritos de cada usuario.	Pantalla donde se gestiona el perfil del usuario.	Muestra el uso de la cámara del dispositivo.
		

Conclusión

La creación de una aplicación utilizando react native ha sido un proyecto que ha permitido explorar y aprovechar al máximo las capacidades de esta tecnología y las habilidades de programación adquiridas durante los tres años de formación en el bachillerato. El objetivo fue desarrollar una aplicación que no solo facilitara el proceso de reserva para los usuarios, sino que también ofrezca una experiencia agradable para los usuarios.

A lo largo del desarrollo del proyecto se presentaron problemas a los que se les tuvo que buscar solución rápidamente para la integración del proyecto, sin embargo, cada obstáculo presentado fue de ayuda para saber que siempre se debe poner el doble de esfuerzo para hacer las cosas y que finalmente se presente un trabajo de calidad.

El uso de una tecnología como React native permitió optimizar los recursos y mejorar la eficiencia del desarrollo de la aplicación para lograr entregarla dentro del periodo de tiempo establecido. El uso de código base para diferentes plataformas no solo redujo el tiempo de desarrollo, sino que también facilitó el mantenimiento y la implementación de nuevas funcionalidades. Además, el apoyo en tecnologías, foros y documentación oficial también fue de gran ayuda para facilitar y resolver problemas de desarrollo.

Se considera que la aplicación creada cumple completamente las necesidades de los usuarios proporcionando una aplicación efectiva y eficiente y aunque el desarrollo de la aplicación no fue del todo fácil, se siente un grado de satisfacción al haber concluido el sexto semestre de bachillerato con un proyecto de esta magnitud que utiliza conceptos básicos que se fueron adquiriendo desde primer semestre. Es fundamental reconocer que sin el apoyo de las profesoras que compartieron sus conocimientos con el grupo, no se hubiera podido lograr nada de lo ya mencionado.

Bibliografía

Bottom tabs navigator. (s/f). Reactnavigation.org. Recuperado el 20 de abril de 2024, de https://reactnavigation.org/docs/bottom-tab-navigator/

Documentación de Firebase (s/f). Recuperado el 20 de abril de 2024, de https://firebase.google.com/docs?hl=es-419

Expo documentation. (s/f). Expo Documentation. Recuperado el 20 de abril de 2024, de https://docs.expo.dev/

React Navigation. (s/f). Reactnavigation.org. Recuperado el 20 de abril de 2024, de https://reactnavigation.org/docs/getting-started/

*React nativ*e. (s/f). Reactnative.dev. Recuperado el 20 de abril de 2024, de https://reactnative.dev/